Adaptive Caching of Spatial Queries in PostGIS Using H3-Based Indexing

Mohammed Shakir
mohsha-0@student.ltu.se
Department of Computer Science, Electrical and Space Engineering

March 24, 2026

Abstract

Web mapping services let users explore and query geographic datasets, for example by requesting vector features inside a map view. In many deployments, GeoServer (a server that publishes geospatial data through standard OGC web services) exposes standard web APIs on top of a PostGIS database. However, when traffic becomes concentrated to a few popular areas (urban hotspots), performance can degrade even with spatial indexes, because many requests overlap and repeatedly trigger similar database work, increasing latency and backend load. This thesis develops an approach for building high-load, interactive vector query services on top of GeoServer/PostGIS by adding an adaptive middleware cache. The approach is based on partitioning each request footprint into H3 hexagonal cells, caching per-cell results in Redis, and maintaining freshness through per-key Time-to-live (TTL) combined with Kafka-driven invalidation. The main idea is to use spatial locality under skew: when many requests overlap the same cells, cached cell payloads can be reused and composed into an exact response while reapplying the original spatial and attribute filters. The proposed methods are implemented as a prototype Go-based middleware.

The prototype is evaluated in a containerized testbed under Zipf-like skew at 800 requests/s, comparing against a no-cache baseline using latency percentiles, throughput, and backend resource usage.

Results show that, when traffic is concentrated in a few hotspot areas, the cache makes the system about 6x faster in the slow cases and cuts PostGIS CPU by about 10x, while still keeping up with the load. The main trade-off is increased memory usage and composition overhead in the caching path, especially when requests span many cells.

Acknowledgements

I would like to thank my internal supervisor and examiner at Luleå University of Technology (LTU), Tatiana Liakh, for steady guidance, clear feedback, and for helping me keep the scope realistic.

I am especially grateful to my external supervisor at Softhouse, Axel Lassinantti, whose advice, reviews, and day-to-day help shaped the project; this thesis would not have come together without his support.

I also thank Metria AB for hosting the work and providing access to their platform and data. In particular, I am grateful to Erik Wikström at Metria for preparing datasets, answering many practical questions, and reviewing intermediate results throughout the project. Softhouse deserves thanks for connecting me with Metria and for offering a great workspace during the project.

Finally, I want to thank my classmates and friends, Elliot Wallin, Suad Huseynli, and Magnus Stenfelt, for carefully reading drafts and providing feedback.

Contents

Abbreviations & Acronyms

ACT Adaptive Cell Trie 12

ARC Adaptive Replacement Cache 18

BBOX Bounding Box . . . . . 14, 16 f., 26 f., 55

CDC Change Data Capture . . . . . 8, 10, 14, 19 f., 54 ff.

ECQL Extended Common Query Language 14, 18, 27

FES Filter Encoding Standard 14, 18

GiST Generalized Search Tree . . . . . 8, 15 f., 52

LSN Log Sequence Number (PostgreSQL WAL position) 19, 21, 28

M/M/1 Single-server queueing model (Poisson arrivals / exponential service) 22

OGC Open Geospatial Consortium . . . . . 8 f., 12, 14

RPS Requests per second . . . 25, 29, 39, 42, 47, 50, 52 f., 55 f.

SAC Semantic Adaptive Caching 12

SRE Site Reliability Engineering . . . . . 21, 24 f.

SRID Spatial Reference System Identifier 15, 25

TTL Time-to-live . . . . . 2, 8 ff., 13 ff., 18 ff., 23–28, 51–56

WAL Write-ahead log (PostgreSQL) 19, 28

WCS Web Coverage Service 12

WFS Web Feature Service . . . . 8, 12, 14, 25 f., 29, 52, 54, 56

WMS Web Map Service 8, 14, 25, 29, 54, 56

WMTS Web Map Tile Service 8

1 Introduction

1.1 Background

Operational web mapping stacks at geospatial service providers typically combine GeoServer for Open Geospatial Consortium (OGC)-compliant services with a PostGIS spatial database. Raster map tiles can be served via Web Map Service (WMS)/Web Map Tile Service (WMTS) endpoints, which use predefined tile matrices to improve the delivery of rendered images [1]. In contrast, vector feature access through Web Feature Service (WFS) exposes fine-grained queries over feature properties and geometries, enabling filtering, editing, and analytics workflows that do not align cleanly with fixed tile boundaries [2]. On the database side, PostGIS provides spatial types and functions with Generalized Search Tree (GiST)-based indexes that are important for performance [3]; yet when traffic is highly skewed toward a few urban "hotspot" regions, many requests repeatedly overlap in space and time, re-triggering similar queries and I/O.

Conventional server-side caching (e.g., tile caches) mostly targets rasterized responses; it is less effective for vector queries with complex geometries, attribute filters, or dynamic layers. A better approach is to partition the query footprint into a spatial grid and reuse results for frequently accessed cells. Hexagonal hierarchical grids such as H3 provide near-uniform cell areas across resolutions, consistent neighbors, and identifiers that are convenient as cache keys and for aggregation across multiple zoom levels.

A spatial cache must also stay fresh because upstream data changes. Time-to-live (TTL) expiration gives a simple baseline, but event-driven invalidation, common in change-data-capture (Change Data Capture (CDC)) pipelines built around streaming platforms, can invalidate only the affected spatial cells when features are updated. Combining TTL with event-driven invalidation improves freshness without discarding useful cache entries. Finally, an in-memory store such as Redis provides the fast access needed to reduce end-to-end latency on cache hits while keeping implementation complexity manageable within a middleware layer. Building on these observations, this thesis presents, implements, and evaluates a Go-based middleware cache between GeoServer and PostGIS. The middleware reduces repeated work for overlapping vector queries in hotspot regions by caching per-H3-cell results in Redis, while keeping cached data fresh using per-key TTL expiration and Kafka-driven spatial invalidation.

1.2 Motivation

Workloads in operational web mapping services frequently show spatial skew: a small set of geographic areas attracts the majority of traffic, while large regions are queried infrequently. Recomputing overlapping vector queries in such hotspots increases latency and worsens database load even when spatial indexes are present. An H3-guided middleware cache provides three benefits:

1. Lower latency by serving popular regions from memory.
2. Lower PostGIS CPU and I/O by avoiding redundant evaluations.
3. Tunable trade-offs between hit ratio, memory footprint, and freshness by adjusting H3 resolution and invalidation policy.

From a research perspective, this work bridges ideas from hierarchical spatial indexing, adaptive caching, and spatially scoped invalidation into a deployable component that complements rather than replaces existing OGC services. The work results in an end-to-end method for evaluating when adaptive caching is worthwhile for dynamic vector workloads, how H3 resolution affects reuse and composition overhead, how to size the cache working set in Redis, and how to integrate time-based freshness (TTL) with Kafka-driven spatial invalidation. The study also contributes a reproducible benchmark and empirical evidence for H3-keyed caching in a setting that is less well served by conventional tile caches and has few publicly documented, end-to-end evaluations.

1.3 Problem Statement

Modern spatial data platforms serve diverse read workloads through GeoServer and PostGIS. In practice, user traffic is highly skewed; a small set of geographic areas (e.g., city centers) receives a disproportionate share of requests, while large regions are rarely accessed. Under this skew, repeated evaluation of overlapping spatial queries increases latency, drives unnecessary CPU and I/O load in PostGIS, and may slow down downstream map rendering. Conventional tile-oriented caching helps for pre-rendered map tiles, but is less effective for dynamic vector queries, highly detailed filters, and requests that do not align with fixed tile boundaries. As a result, the platform needs a cache that adapts to *where* and *how often* users query, while remaining correct and fresh when data changes.

Thus, the main goal of this thesis is to develop, implement, and evaluate an *adaptive, spatially aware* caching layer between GeoServer and PostGIS that uses an H3 hexagonal grid to capture hotspot locality, reuses query results from Redis, and maintains acceptable freshness through a combination of time-to-live (TTL) and event-driven invalidation via Kafka. The goal is to reduce end-to-end query latency and database load under skewed workloads without introducing unacceptable staleness or excessive memory overhead.

To achieve this goal, the following tasks were set:

1. Investigate how existing solutions handle caching and freshness for dynamic vector queries in GeoServer/PostGIS-style stacks, and derive requirements for hotspot-heavy workloads.

2. Design an H3-keyed caching approach for vector results, including keying, composition, and correctness safeguards for spatial/attribute filtering.

3. Implement a Go-based middleware prototype with Redis-backed per-cell caching and Kafka-driven spatial invalidation.

4. Experimentally evaluate the prototype under controlled skewed workloads, comparing latency percentiles, throughput stability, backend load, memory footprint, and freshness implications across H3 resolutions and policies.

1.4 Research Questions

To evaluate the proposed approach, this thesis investigates the following research questions:

1. **Existing solutions and gap:** How do existing caching approaches for GeoServer/PostGIS-style vector services handle spatial overlap reuse and cache freshness, and what limitations remain for hotspot-heavy, dynamic vector workloads?

2. **Effectiveness:** To what extent does H3-based adaptive caching reduce latency and offload query pressure from PostGIS compared to no caching or a simple geometry/TTL cache baseline under skewed workloads?

3. **Granularity:** Which H3 resolution(s) give the best balance between cache hit ratio, memory footprint, and data freshness, given uneven access to different areas?

4. **Invalidation Policies:** What combination of time-to-live (TTL) and event-driven invalidation (triggered by upstream data change notifications) provides acceptable staleness bounds while keeping high performance?

1.5 Scope

This thesis carried out the tasks defined in the problem statement by (i) reviewing how existing GeoServer/PostGIS-style stacks approach caching and freshness for dynamic vector queries and deriving requirements for hotspot-heavy workloads, (ii) designing an H3-keyed caching method for vector results, including cache keying, multi-cell composition, and safeguards to preserve correctness under spatial and attribute filtering, (iii) implementing a Go-based middleware prototype that intercepted GeoServer requests and integrated Redis for per-cell caching together with TTL-based expiry and Kafka/CDC-driven spatial invalidation, and (iv) experimentally evaluating the prototype in a containerized testbed under synthetic, skewed workloads representative of urban hotspots.

The evaluation quantified end-to-end latency percentiles, throughput stability and error rates, backend load (notably PostGIS and GeoServer CPU/memory), and cache behavior (reuse indicators, memory/evictions, and freshness implications) across selected H3 resolutions and freshness policies, emphasizing the trade-offs between spatial granularity, composition overhead, cache memory footprint, and bounded staleness.

1.6 Thesis Structure

The thesis is organized as follows:

- **Introduction** outlines the background, motivation, problem statement, research questions, scope, contributions, and thesis structure.

- **State of the Art** surveys prior research on spatial indexing, adaptive caching, and cache consistency for dynamic geodata, and also introduces the theoretical background needed to understand H3 indexing, caching principles, and consistency mechanisms.

- **Method** describes the H3-based request bucketing, hotness tracking, cache invalidation policies, and the middleware design.

- **Verification and Validation** reports the detailed measurements, stability checks, and backend resource evidence that supports the results.

- **Result** presents the main empirical outcomes and headline numbers.
- **Discussion** reflects on the results, limitations, and threats to validity and includes considerations of sustainability.
- **Conclusions** summarize the contributions of the thesis and outline directions for future improvements and deployment.

2 State of the Art

2.1 Related Work

This section frames the thesis in four strands of prior work:

- Hierarchical spatial indexing and pre-aggregation.
- Adaptive caching for spatial workloads.
- Cache consistency and invalidation with spatial scope.
- Hexagonal grid indexing (H3).

The goal is to motivate the chosen H3-guided middleware design and clarify how it differs from existing systems.

2.1.1 Hierarchical spatial indexing and pre-aggregation

A central idea behind the proposed middleware is to divide query footprints into grid cells so that repeated queries over similar regions can be identified and reused. Work on *GeoBlocks* pre-aggregates point data into fine-grained cells and approximates query polygons by unions of those cells. The approximation error is bounded by the chosen cell size, and a trie-like cache reuses previous results for frequently queried regions, adapting to skew over time [4]. Similarly, *Adaptive Cell Trie (Adaptive Cell Trie (ACT))* speeds up point-in-polygon joins by combining multiresolution (quadtree-style) approximations with a radix-tree over cell identifiers; the design avoids most expensive geometric tests while offering user controlled precision [5]. Earlier systems such as *Nanocubes* and the *aR-tree* introduced in-memory spatio-temporal aggregation over hierarchical rectangles; they offer interactive performance but rely on rectangular partitions that can cause unbounded error for arbitrary polygons unless cell size is carefully managed [6, 7]. Together, these works support a grid-based approach that supports multiresolution refinement near boundaries, exposes compact cell identifiers for fast lookup and reuse, and adapts well under workload skew properties leveraged with H3-based bucketing.

2.1.2 Adaptive caching for spatial workloads

Beyond indexing, several systems adapt caching to hotspot access patterns. *STASH* is a distributed, in-memory middleware cache for hierarchical aggregations that chooses replicas based on query frequency and freshness, leading to large latency reductions and a higher query processing rate under skew [8]. *GeoBlocks* cache also adapts automatically as regions get popular [4]. At the opposite end of the stack, *Semantic Adaptive Caching (SAC)* (Semantic Adaptive Caching) predicts future query windows on the client using partitioned spatial history and prefetches answers to improve responsiveness [9]. Finally, *Vecstra* proposes an OGC-compliant in-memory geospatial cache that integrates caching with standard WFS/Web Coverage Service (WCS) interfaces, which is useful when the server side must remain standards based, as in GeoServer/PostGIS deployments [10]. Although not spatial per se, *MinMaxCache* shows how an

adaptive in-memory cache can trade the level of aggregation for interactive visualization latency with error, an idea that inspires the evaluation of H3 resolution choices and composition overheads in this thesis [11]. The common thread is that caching should be driven by query semantics, popularity, and acceptable approximation, rather than being a static tile store; the middleware follows this line by caching vector results keyed by H3 cells and adapting to observed hotness.

2.1.3 Cache consistency and spatially scoped invalidation

Keeping cached results fresh is not easy when data and query scopes are spatial. Classic mobile computing studies introduced *location dependent* invalidation: cached answers can become stale not only because the data change but also because the user moves outside a valid geographic scope [12]. This led to approaches like *Bit-Vector* with *Compression and Implicit Scope Information*, which link cached items directly to their spatial validity regions [12]. Related work on *location dependent semantic caching* also organizes cache segments with explicit spatial scopes and uses them in replacement and validation [13]. The practical takeaway for a server-side cache is to tie each cached entry to an explicit spatial footprint and to invalidate it on time-based expiry (TTL) and spatial overlap with changed features. The proposed design does exactly this: H3 cell keys define validity scopes, while TTL and Kafka-driven events provide temporal and event based invalidation hooks.

2.1.4 Hexagonal grids and H3

H3 is a hierarchical, global grid of hexagonal cells. Its properties include near-uniform area per resolution, consistent adjacency, and a clean hierarchical relation between resolutions that make it attractive for bucketing and composing spatial answers [14]. Empirical reports outside pure research also suggest that hexagons reduce visual and aggregation artifacts compared to rectangular geohashes for many analytics tasks [15]. For our use case, H3 gives a compact key that is independent of the programming language (suitable for Redis), deterministic up/down-sampling across resolutions for composition, and a natural way to map data change events to affected cells for invalidation.

2.1.5 Gap analysis

The surveyed literature provides several of the building blocks needed for hotspot-heavy vector workloads, but it tends to address them in isolation. Grid/index systems such as GeoBlocks and ACT focus on accelerating specific query classes or pre-aggregation and do not present a middleware cache for standards-based GeoServer/PostGIS vector serving with targeted, event-driven freshness [4, 5]. STASH demonstrates strong benefits from adaptive middleware caching, but targets hierarchical aggregation workloads rather than caching standards-based vector feature responses with fine-grained spatial invalidation [8]. Client-centric approaches such as SAC improve perceived responsiveness via prediction and prefetching, but operate on the client side and do not reduce server-side recomputation in the same way [9]. Vecstra discusses an OGC-compliant caching architecture, but does not emphasize cell-level invalidation tied to streaming updates for only the affected spatial scopes [10].

Table 1: Qualitative comparison of the closest related work against the thesis contributions. "Partial" indicates that the aspect exists but differs materially in scope (e.g., aggregates instead of vector feature responses, or invalidation without fine-grained spatial scoping).

Work	Middleware cache	Vector cache	Adaptive	Schema agnostic	Res. eval.	Event-driven freshness	OGC focus
GeoBlocks [4]	No	Partial	Yes	Depends	Partial	No	No
ACT [5]	No	No	N/A	N/A	Partial	No	No
STASH [8]	Yes	Partial	Yes	Depends	Partial	Partial	No
SAC [9]	No	Yes	Yes	Depends	No	No	Partial
Vecstra [10]	Yes	Yes	Partial	Yes	Partial	Partial	Yes
This thesis	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

Against this background, this thesis contributes an end-to-end prototype and evaluation that combines a query-driven hotspot identification by bucketing arbitrary WFS/WMS footprints into H3 cells outside the database, adaptive per-cell caching of vector query results in a middleware layer with measured trade-offs across H3 resolutions, and freshness via TTL together with event-driven invalidation using Kafka/CDC for the overlapping H3 cells. Table 1 summarizes how these aspects relate to the closest prior work (based on the published descriptions of each system).

2.2 Theory

This section provides background theory for the design and evaluation choices made in this thesis; it does not describe an external system developed by others. It introduces the concepts that the *proposed middleware cache* relies on, including how vector queries are expressed and scoped, why real workloads concentrate on a few "hot" areas, and how spatial indexes in PostGIS speed up individual queries yet still leave room for reuse through caching. The aim is to establish consistent terminology and assumptions so that later *thesis contributions* (in particular the middleware design, H3 resolution choices, cache keys, and freshness policies) can be clearly understood and compared.

2.2.1 Conceptual Frame

2.2.1.1 Query Models and Spatial Footprints

Vector web services define two main types of query filters: a *spatial filter* over geometries and an optional *attribute filter* over feature properties. In OGC-style services, the spatial part ranges from coarse bounding boxes (Bounding Box (BBOX)) to polygonal predicates such as `Intersects`, `Within`, or `Contains`, while the attribute part uses comparison and logical operators (e.g., `POP_DENSITY > 5000 AND TYPE = 'residential'`). Filter Encoding (Filter Encoding Standard (FES)) defines how these spatial, comparison, and logical operators are combined into a single query expression, in either XML or Extended Common Query Language (ECQL) form [16].

The term spatial footprint refers to the minimal geometry defining the spatial filter sent by the client (e.g., the BBOX or a user-drawn polygon). For caching theory, the footprint is mapped to a distinct set of spatial buckets (H3 cells) that act as reusable containers. A *spatial response* is the set (or stream) of

features matching both filters within the query area, together with any server-side transformations. Reuse then follows two patterns:

- Full reuse: When a new request's cell set and attribute filter exactly match a cached entry.
- Composed reuse: When the new request spans multiple cached cell entries whose combination can be merged and then filtered.

In both cases, the cache key must encode (layer, cell or cell-set, filter hash, Spatial Reference System Identifier (SRID)) to preserve identical query meaning with the database result. Freshness is controlled via TTL or event-driven invalidation (discussed later), but the scope of what to invalidate remains the same, which is the cells touched by the updated features.

2.2.1.2 Hotspots and Workload Skew

Interactive map traffic is rarely uniform across space; request popularity typically follows a heavy-tailed (Zipf-like) distribution, where a small number of locations attract a large fraction of requests, while most locations are cold. This pattern is common in many networked systems and motivates cache designs that focus on retaining popular items. In spatial systems, the same pattern appears as *spatial skew*: urban centres, transport corridors, and tourist areas become hotspots, and the set of hotspots may evolve over time with events and daily cycles [17].

Skew has two practical consequences. First, repeated evaluation of overlapping queries in hotspots can dominate query latency and backend load, even when individual queries are optimized with spatial indexes. Second, any effective cache should be *spatially selective*, meaning that it should spend memory on the small subset of cells where reuse is high, rather than caching uniformly across the entire map. Dividing footprints into H3 cells makes it possible to track per-cell popularity, admit only "hot" cells, and combine results for larger queries without caching redundant geometry everywhere.

2.2.2 Spatial Indexing and Partitioning

2.2.2.1 Spatial Indexes in PostGIS

PostGIS optimizes spatial queries using GiST-based indexes that implement an R-tree-like search over geometry bounding boxes. The index performs a fast filter step (bounding-box test) to filter out candidates, followed by an exact geometry check (`ST_Intersects`, `ST_Within`, etc.) on the survivors [3]. Correct use therefore requires both creating the index with `USING GIST` and using index-aware filters so the planner can apply the filter step.

While GiST dramatically reduces the cost of single queries and spatial joins, it does not take advantage of *temporal locality* or *overlap reuse*. If many users repeatedly query the same small region, the database still performs planning, index descent, and exact checks for each request; buffers, caches, and prepared geometries help, but they are not keyed by *where* reuse occurs. An external, cell-keyed cache complements GiST by avoiding the entire evaluation pipeline for popular regions. Once a cell's result is computed, later requests that map to the same (cell, filter) can be answered from memory. In short, GiST reduces the work needed to find candidates, while spatial caching reduces how often the database has to perform that work at all.

2.2.2.2 Grid-Based Bucketing vs. Native Indexing

Native database indexing (e.g., GiST-backed R-trees in PostGIS) speeds up single queries by filtering out candidates with a fast bounding-box filter and then applying exact geometry filters to the survivors. This *two-pass* evaluation is highly effective per request, but it does not directly capture *temporal locality* across requests that repeatedly target the same small area: the planner, index descent, and exact checks still run for every query.

An external, grid-based bucketing strategy divides space into fixed cells and uses those cells as cache keys for query answers. In this model, a request's footprint (BBOX or polygon) is mapped to a set of cells; if the (layer, cell-set, filter) tuple is already cached, the middleware composes the response from memory, bypassing the database. Overlapping requests can then reuse previous results because they share cells [4]. The database remains responsible for correctness and misses, while the middleware focuses on detecting and serving hotspots. In short, native indexing reduces the amount of work within a query, while grid bucketing reduces how often the database must do that work at all, by enabling answer reuse across partially overlapping requests. The two techniques complement each other: GiST handles precise geometry tests and complex joins, while grid bucketing leverages spatial clustering to reduce repeated work in hotspot regions.

2.2.2.3 Hexagonal Grids and H3 Basics

H3 is a hierarchical discrete global grid that represents locations as 64-bit cell identifiers. The grid is organized in resolutions ranging from 0 to 15, where each cell has exactly one parent at any lower resolution and up to seven children at the next higher resolution; parent/child navigation is provided by API functions such as `cellToParent` and `cellToChildren` [18]. Adjacency is based on shared edges; neighborhood queries use grid traversal operations to list cells within a given graph distance.

At resolution 0, the grid contains 122 base cells (derived from an icosahedral projection); refinements add hexagonal children, with pentagons appearing to maintain topology [19]. These properties (stable adjacency, clean up/down sampling, and compact integer keys) make H3 a practical foundation for grouping requests and assembling answers across neighboring cells in a middleware cache.

2.2.2.4 H3 Resolutions and Cell Geometry

Cell size decreases exponentially with resolution. Average hexagon areas are on the order of $5.16\,\mathrm{km}^2$ at res 7, $0.737\,\mathrm{km}^2$ at res 8, and $0.105\,\mathrm{km}^2$ at res 9 (these figures represent global averages, though the exact area depends on each cell's position relative to the icosahedral faces) [20]. The minimum/maximum area ranges per resolution reflect distortion near icosahedron vertices and the presence of pentagons.

These values give practical trade-offs. Higher resolutions improve cache specificity (higher chance that an incoming query is covered by already-hot cells and less overfetch when assembling answers) but increase:

- The number of keys per query.
- Metadata/memory overhead per cached entry.
- Composition cost when merging many cell payloads.

Lower resolutions reduce key churn and memory but risk overestimation (returning many features outside the exact footprint) and lower hit probability for small, detailed queries. The evaluation therefore considers a small set of candidate resolutions and reports hit ratio, composition overhead, and memory per resolution, based on documented cell-area curves.

2.2.2.5 Mapping BBoxes/Polygons to Cells

Request footprints are consistently converted into H3 cell sets. A rectangular BBOX is treated as a four-corner polygon, while user-defined polygons are used directly. The conversion uses H3's region functions: given a polygon and a target resolution, return the set of cell IDs whose centroids lie inside the polygon [21]. This centroid-containment rule is explicit in both the H3 reference and in SQL integrations. The inverse operation reconstructs polygon outlines from a set of cells, which is used for debugging and visualization.

Boundary effects must be handled carefully. Because containment is based on cell centroids, narrow polygons may miss edge-touching cells whose centers fall just outside, while low-resolution cells can over-cover the footprint. These issues are handled by choosing a resolution appropriate to the query's spatial detail, optionally expanding the boundary by a 1-ring and clipping results to the original geometry. For multipolygons and holes, the outer ring and interior rings are passed to the region function so that coverage excludes holes and unconnected islands are handled correctly. Neighboring cells are included only when a buffer around the footprint is explicitly needed for later rendering or prefetching [22].

2.2.2.6 Spatial Approximation Limits of H3

Converting continuous geometries into hexagonal cells introduces approximation errors that must be considered during design and evaluation. First, *centroid-based coverage* in common `polygonToCells`/`polyfill` routines only includes cells whose centres fall inside the polygon [21]; thin corridors or sliver-like shapes near boundaries can therefore be under-covered unless the resolution is increased or a slight expansion step is added. Second, the H3 grid is constructed on an icosahedral projection, which gives small but systematic *edge misalignments* and area variation across the globe, cells near certain parts of the icosahedron deviate more from equal-area assumptions [23], and reconstructed boundaries from cell sets may appear slightly jagged compared to the original geometry. Third, *pentagon distortion* creates uncommon neighbourhoods where rings/ranges have special handling, algorithms that rely on uniform six-neighbour topology can fail or require slower fallback paths around pentagons [22].

2.2.3 Caching Theory for Spatial Workloads

2.2.3.1 Caching Objectives and Cost Model

The cache objectives are defined in terms of latency and backend offload. Let H be the cache hit ratio for a given configuration (resolution, admission, replacement). Let L_{hit} be the end-to-end latency on a hit (middleware lookup + composition + optional post-filter), and L_{miss} the latency on a miss (database + network + rendering). The expected latency is

$$\mathbb{E}[L] = H\,L_{\text{hit}} + (1 - H)\,L_{\text{miss}}.$$

Similarly, the backend work (e.g., CPU/IO in PostGIS) is modeled with per-request costs B_{hit} and B_{miss} (typically $B_{\text{hit}} \ll B_{\text{miss}}$), giving

$$\mathbb{E}[B] = H\, B_{\text{hit}} + (1 - H)\, B_{\text{miss}}.$$

A simple overall objective can be expressed as a weighted sum

$$J = \alpha\, \mathbb{E}[L] + \beta\, \mathbb{E}[B] + \gamma\, S,$$

where S measures staleness risk (e.g., fraction of responses older than a freshness target) and (α, β, γ) reflect deployment priorities. In practice, the tail latency (P95/P99) is also reported, because heavy-tailed request sizes and composition can make percentiles differ significantly from the mean [24]. Under this model, policies are compared by how they trade hit ratio and composition overhead against memory footprint, while respecting a freshness bound set by TTL and event-driven invalidation.

2.2.3.2 Key Design for Spatial Queries

Cache keys define how granular and isolated cache reuse is. A composite key that captures all the information needed is used to guarantee matches with a database answer:

`key` = `<layer>` : `<srid>` : `<resolution>` : `<cellId>` :
`<filterHash>` : `<version>`.

Here `layer` identifies the dataset; `srid` fixes the coordinate space; `resolution`/`cellId` identify the H3 bucket; `filterHash` fingerprints the attribute condition (normalised ECQL/FES); and `version` encodes schema/data epoch for bulk invalidation.

This per-cell design avoids generating a large number of unique keys for complex polygons, improves reuse because nearby cells often appear in many queries, and allows targeted invalidation when upstream changes touch only a subset of cells. Larger "query keys" (entire cell-sets) can be layered on top as an optimisation for extremely frequent, identical windows, but the base unit of reuse remains the single cell.

2.2.3.3 Admission and Replacement

Since spatial workloads are uneven, the cache should prioritise cells that remain popular over time while avoiding frequent replacements. A hotness-based admission rule is used: each cell keeps a fading frequency counter, and new items are stored only if recent access passes a set limit or if they replace a clearly less active item already in the cache. Lightweight frequency estimators (for example, TinyLFU-style) give small, memory-efficient estimates that work well under Zipf-like access patterns.

For replacement, both how recent and how frequent accesses are considered to protect cells that are either short-term spikes or long-term hotspots. In practice, an Adaptive Replacement Cache (ARC)-style split between recent and frequent lists adapts to changing traffic patterns [25], while the admission rule prevents one-time scans from filling up the cache. Spatially aware strategies include:

- Group removal: when memory becomes limited, remove clusters of nearby, cold cells at the same resolution to avoid uneven gaps.

- Resolution aware fallback: before removing many small, high-detail cells, try combining them into a larger cell if it still provides good reuse.
- Ghost records (metadata only): that remember removed cells to guide future admission choices.

These methods keep the active cache focused on real hotspots and help the hit rate stay steady even when demand patterns change.

2.2.3.4 TTL-Based Freshness Control

A time-to-live (TTL) sets an upper bound on how long a cached cell may be reused without revalidation. TTL follows the same idea as the HTTP *freshness lifetime*: a response is *fresh* while its age is below the set limit and becomes *stale* afterward. In the middleware, each Redis entry stores its own TTL, and expiration is handled by Redis through active/idle checks and lazy deletion on access. Let τ be the TTL and assume upstream edits arrive as a Poisson process with rate λ for a given cell. Then the probability that a cached answer is stale at access time is $P_{\text{stale}} = 1 - e^{-\lambda\tau}$ [26], which makes the $\lambda\tau$ product a practical tuning knob, where larger τ increases hit ratio but also the risk of staleness.

Default TTLs are set per layer, and adaptive TTLs are supported: shorter τ values are used for frequently updated layers (e.g., work orders) and longer ones for static layers (e.g., base maps). To cap worst-case staleness, clients can request *fresh* responses by bypassing the cache or forcing revalidation. Finally, TTL interacts with event-driven invalidation, so any update event that maps to a cell immediately removes or refreshes the entry, regardless of remaining TTL (see next subsection).

2.2.3.5 Event-Driven Invalidation

TTL alone cannot guarantee up-to-date data for regions that change often. As a result, the system subscribes to change-data-capture (CDC) streams from PostgreSQL via Debezium, which tails the database Write-ahead log (PostgreSQL) (WAL) and sends an event per `INSERT`/`UPDATE`/`DELETE` to Kafka topics [27]. Each event includes table and row identifiers, and when geometries change, the middleware calculates which H3 cells are affected and invalidates their corresponding cache keys. This gives near-real-time consistency for updated features while avoiding global cache purges.

Because streaming systems provide ordering only within a partition, events are partitioned by stable identifiers (e.g., feature ID), so that all changes for the same feature arrive in order [28]; across features, out-of-order delivery is acceptable because invalidation is repeat-safe. Repeat-safe logic is implemented with a per-key *version/epoch* based on the source commit Log Sequence Number (PostgreSQL WAL position) (LSN); invalidating the same cell twice is safe, and late events with older LSNs are ignored. Delivery guarantees are at-least-once by default, so when needed, Kafka's repeat-safe producers/transactions enable exactly-once processing in consumers, though with higher operational cost. In practice, combining at-least-once delivery with repeat-safe, versioned invalidation is sufficient: duplicates only cause harmless extra removals, and missing entries are simply repopulated on the next access.

2.2.4 Consistency and Freshness Semantics

2.2.4.1 Definitions: Consistency, Coherence, and Staleness

Coherence is a single-object property: all replicas of the same cached item eventually agree on the most recent value; in other words, two readers of the same key do not see conflicting versions once updates have spread [29]. *Consistency* refers to multi-object and temporal guarantees, e.g., whether reads reflect a globally ordered history of writes. Strong models such as linearizability make each operation appear atomic in real time [30], whereas weaker models (e.g., eventual consistency) allow temporary differences [31]. The middleware targets *bounded staleness* rather than full linearizability: bounded in time using TTL, and shortened with event-driven invalidation.

Staleness measures how "out of date" a cache response is relative to the origin. A response becomes *stale* once its age exceeds its freshness lifetime. Staleness is reported as a fraction of responses served beyond their freshness targets, and policies are preferred that keep this fraction below a layer-specific threshold while still delivering high hit ratios.

2.2.4.2 Spatial Validity Scopes

In spatial workloads, every cached entry has to carry an explicit validity *scope*. Each value is tied to the H3 cell (or cell set) used as its key, and an update is relevant only if the updated geometry intersects that scope. This mirrors prior work on location-dependent caching, where invalidation combines temporal and spatial conditions to avoid discarding unrelated data. Formally, let $V(k)$ be the validity polygon of key k (the cell footprint), and let Δ be the geometry delta from an update event. Invalidate k when $V(k) \cap \Delta \neq \varnothing$.

Two edge cases matter in practice. First, boundary cells: to reduce false negatives due to discretization, the scope can be expanded by a one-ring buffer at the chosen resolution and the exact PostGIS filter can still be reapplied at compose time. Second, multi-cell requests: scopes compose as unions; invalidation touches only the intersecting cells, keeping cache content for unaffected neighbors. Together with TTL, these spatial scopes limit cache clearing and keep popular regions updated without emptying the entire cache.

2.2.4.3 Combining TTL with Event-Driven Updates

A hybrid freshness strategy combines a *time-to-live* (TTL) with *event-driven* invalidation so that cached entries are reused aggressively yet replaced quickly when underlying data change. TTL provides a deterministic upper bound on age: a value is *fresh* while its age $< \tau$ and *stale* after that [32]. Event-driven invalidation reacts to specific updates (e.g., CDC messages) by removing or refreshing only the keys whose spatial scopes intersect the changed features. Let updates for a particular cell arrive according to a rate λ, and let D be the end-to-end delay from an upstream commit to the consumer performing invalidation. The worst-case staleness window is $\min(\tau, D)$, and the approximate probability that a randomly timed read will encounter stale data is $1 - e^{-\lambda \min(\tau, \mathbb{E}[D])}$ [26]. Tuning is therefore done along two axes:

- Choosing τ by layer volatility and tolerance for stale reads.
- Reducing $\mathbb{E}[D]$ by provisioning streaming paths and consumers.

In practice, *refresh-on-read* for near-expiry hits is also included, a *write-through* path for updates made through the service itself, and short *grace* windows that allow slightly stale entries to be served while a background refresh runs, keeping tail latency low while still controlling staleness.

2.2.4.4 Delivery Semantics and Ordering

Event-driven invalidation relies on the messaging substrate's guarantees. Kafka keeps order *within* a topic partition, so partitioning by a stable key (e.g., feature identifier or H3 cell) gives in-order change events for items mapped to the same key [28]. Default delivery is at least once, so consumers must therefore be repeat-safe. An ever-increasing source version (e.g., PostgreSQL LSN from Debezium) is attached to each event and *compare-and-set* invalidation is applied: drop cache entries if and only if the incoming version is newer than the last processed one. Duplicate or out-of-order events are therefore harmless [27]. Where required, Kafka's reliable producers and transactions enable exactly-once processing for read-process-write topologies, trading operational complexity for stronger guarantees. Finally, recovery time after outages is limited by replaying from a saved offset and reapplying repeat-safe invalidations. Because invalidation is repeat-safe, replay restores consistency without global cache purges. Together, partitioned ordering, repeat-safe handlers, and version checks ensure a consistent, minimal-invalidations stream that keeps the cache aligned.

2.2.5 Performance and Latency Modeling

2.2.5.1 End-to-End Latency Decomposition

End-to-end latency is broken down into separate components to identify bottlenecks and target optimizations. Let a request traverse:

- The client → middleware network.
- Middleware parsing/routing (T_{mw}).
- Cache lookup and (if hit) result composition plus optional post-filter.
- Database path on a miss—planner, index traversal, predicate evaluation, result materialization.
- Middleware → client transfer plus serialization.

Then the total end-to-end latency T_{e2e} is

$$T_{\text{e2e}} = T_{\text{net,in}} + T_{\text{mw}} + \begin{cases} T_{\text{cache}} & \text{hit,} \\ T_{\text{db}} + T_{\text{cache(post)}} & \text{miss,} \end{cases} + T_{\text{net,out}}.$$

On hits, $T_{\text{db}} = 0$ and T_{cache} dominates, while on misses, T_{db} typically dominates. Each segment is measured and percentiles (P50/P95/P99) are tracked rather than means, since heavy-tailed effects can hide user-visible slowdowns in averages [24]. Additional Site Reliability Engineering (SRE) "golden signals" (latency, traffic, errors, saturation) help correlate spikes [33] in T_{db} with database saturation, or rises in T_{cache} with composition overhead at very high H3 resolutions. This decomposition helps design experiments (e.g., resolution sweeps) and informs operational decisions such as whether to scale the middleware, database, or network.

2.2.5.2 Queueing Basics and Little's Law

Queueing theory connects arrival rate, concurrency, and delay. Little's Law states that the average number of in-flight requests N equals arrival rate λ times average response time R: $N = \lambda R$ [34]. Therefore, at a fixed λ, reducing R (via cache hits) lowers concurrency and pressure on backend systems. For a single-server station with exponential arrival gaps and service times (Single-server queueing model (Poisson arrivals / exponential service) (M/M/1)), service rate $\mu = 1/S$ (mean service time S) and utilization $\rho = \lambda/\mu$. Stability requires $\rho < 1$, and the expected response time is

$$R = \frac{S}{1-\rho} = \frac{1/\mu}{1-\lambda/\mu}$$

cf. [35]; this shows how delay grows nonlinearly as utilization approaches 1. In this context, the 'server' can represent the PostGIS node on cache misses or the middleware on cache hits; the model can be extended to multiple cores or nodes (e.g., M/M/k). Arrival rate λ is estimated from observed traffic, R is measured per path (hit/miss), and concurrency limits and thread pools are sized so that ρ stays well below overload levels at target latencies. These formulas provide a simple way to plan capacity and highlight the trade-off between raising the hit ratio and improving database speed.

2.2.5.3 Tail Latency and Percentiles

Average latency can hide slow cases that have the biggest impact on how responsive a system feels to users. A small number of slow operations (for example, long network paths, cache misses, garbage collection pauses, or slow database queries) can build up across microservices and cause much longer end-to-end delays. For this reason, high-percentile service-level indicators (SLIs), like P95 and P99, are used to measure interactive workloads. P95 shows how most users experience the system during load spikes, while P99 captures the slowest visible responses under normal conditions and is affected by extra delay added by combining multiple backend calls in one request. These percentiles help guide both design (for example, keeping cache merge time low and limiting how many backends a request touches) and operations (for example, alerting on slowdowns that do not change the average). P50, P95, and P99 are reported for the full path and for key parts (cache, middleware, database).

2.2.5.4 Hit Ratio vs. Latency/Throughput Trade-off

Caching reduces both latency and backend load through two effects: serving hits quickly from memory and not sending those requests to the database. Let H be the hit ratio. With average hit and miss latencies L_{hit} and L_{miss}, the expected end-to-end latency is

$$\mathbb{E}[L] = H\,L_{\text{hit}} + (1-H)\,L_{\text{miss}}.$$

Similarly, the expected backend work (CPU/I/O) is

$$\mathbb{E}[B] = H\,B_{\text{hit}} + (1-H)\,B_{\text{miss}},$$

with $B_{\text{hit}} \ll B_{\text{miss}}$. Increasing H therefore lowers $\mathbb{E}[L]$ and directly offloads the database. Under Little's Law, reducing R (response time) also lowers in-flight concurrency at the backend, improving queueing and tail latency. However,

raising H by pushing to very small spatial cells can add extra merge work and frequent key changes, which raises L_{hit} and memory pressure. The optimal point balances:

- Admission/replacement that prioritise reusable cells.
- Choosing a cell size that keeps reuse high while limiting how many cells each request needs.
- Freshness policies (TTL/events) that avoid invalidating hot cells too aggressively.

The evaluation therefore reports $(H, \mathbb{E}[L], \text{P50/P95/P99}, \mathbb{E}[B])$ together to show these trade-offs.

2.2.5.5 Granularity vs. Memory Cost (H3 Resolution)

H3 offers a hierarchy of resolutions: as the resolution increases, cells become smaller, and the number of cells needed to cover the same area grows roughly exponentially. Finer resolutions give better spatial accuracy—queries overlap more with cached cells, and extra data fetched outside the target area decreases—raising the chance of cache hits for small, detailed regions. The trade-off is more keys per request, higher metadata cost per cell, and longer merge times when combining many cell results. Coarser resolutions show the opposite effect: fewer keys, smaller index structures, and faster merging, but more overcoverage near boundaries and a lower chance that small queries match popular cells. Memory use also grows with the number of stored (layer, resolution, cell, filter) entries; at high resolutions, busy urban areas can spread across many nearby cells unless caching is selective. In practice, a few resolutions are tested and the one that gives the best overall results is chosen, with the highest P50/P95/P99 improvement under acceptable memory use and merge cost. When hotspots cover multiple cells, a coarser "umbrella" entry can also be stored for bursts of traffic, while a subset of fine cells is kept for high-detail queries.

2.2.5.6 Workload Skew and Hotspot Dynamics

Spatial access patterns are usually uneven; a small number of regions get most of the traffic while the rest stay cold. Also, popularity changes over time with events or daily activity cycles. For this reason, per-cell activity is tracked using *time-aware* counters instead of raw counts. A sliding window over recent requests captures short-term demand and reacts quickly to changes, but it may forget too quickly. In contrast, *exponentially decayed* counts adapt more smoothly by giving less weight to older accesses using a decay factor $\alpha \in (0, 1)$. Cells are then admitted when their decayed frequency passes a threshold, compared to their memory cost and merge time. Replacement keeps long-term hotspots (high long-term weight) while letting short-lived spikes use space briefly in the "recent" list. To reduce uneven coverage, mild *spatial smoothing* is applied, where nearby cells inherit a small share of a cell's activity, helping to catch slightly shifted viewports. Finally, to handle sudden bursts (for example, a city-wide event), the number of new cells that can be added in one interval is limited and coarser cells are temporarily promoted, merging many small keys into one while keeping fine-grained entries that stay consistently active.

2.2.5.7 Golden Signals and Cache Metrics

Standard SRE "golden signals" are adopted together with cache-specific metrics for evaluation and operations:

- **Latency:** P50/P95/P99 for end-to-end, cache-hit path, and miss path.
- **Throughput (Traffic):** requests/s overall, hits/s, misses/s; per-layer breakdown.
- **Errors:** non-2xx/5xx rates and cache/middleware exceptions.
- **Saturation:** CPU, memory, and connection pool utilisation for middleware and PostGIS; Redis memory/evictions.
- **Cache ratios:** hit/miss ratio, byte hit ratio, and origin offload (misses avoided).
- **Freshness:** fraction of responses older than target; invalidations/s; average delay from update to invalidate.
- **Spatial stats:** per-cell request share (skew), active-key count, keys per request, and average bytes per cell.

These metrics connect design choices (resolution, admission/replacement, TTL/events) to user outcomes (tail latency) and system health (backend offload and saturation).

3 Method

The method has two parts: (i) design and implement an H3-keyed middleware cache with Redis and Kafka-based invalidation, and (ii) evaluate it under controlled, skewed workloads. Figure 4 first positions the system in its docker-compose testbed context (what talks to what). Figure 1 then shows the concrete architecture and data flows for the read path and the update/invalidation path. Figure 5 summarizes the middleware's internal modules and responsibilities. Figure 2 specifies the request-handling algorithm step-by-step in a way that can be repeated. Finally, Figure 3 summarizes the experimental run procedure.

3.1 Research Approach and Study Design

The goal is to measure how an H3-keyed middleware cache affects end-to-end latency and backend load under skewed spatial workloads, compared to no cache. The evaluated data flows are shown in Figure 1, and the reproducible request-handling procedure is specified in Figures 5 and 2. Each configuration is run with the same dataset and request list (fixed seed), and a scripted procedure is used so runs are repeatable (Figure 3).

For each workload profile, configurations are cycled through, runs are repeated, and median and percentile metrics are reported across repetitions. External factors are fixed as much as possible (hardware, network, versions, dataset, and request list). The system is warmed up, and measurements are collected during a steady-state window. The SRE "golden signals" are reported alongside cache-specific indicators: P50/P95/P99 latency (overall, hit-path, miss-path), requests/s, error rate, saturation (CPU/memory), and full-hit/partial-hit/miss rates.

Independent variables are workload skew (Zipf_s), offered load (Requests per second (RPS)), H3 resolution r, and freshness policy (TTL and invalidation enabled/disabled). The main comparisons in this thesis use $r \in \{7, 8, 9\}$. Dependent variables are latency percentiles, achieved throughput, PostGIS CPU, Redis memory/evictions, and staleness-related indicators.

Validity measures include repeated trials, fixed seeds for traffic, and cross-checking results. Threats to validity are noted (dataset representativeness, synthetic vs. real traffic, and Docker networking differences), and the evaluation is framed as comparative rather than absolute.

3.2 System Under Study and Assumptions

The evaluated stack combines standard components with a Go middleware proxy developed in this thesis. Figure 4 shows the docker-compose testbed at container level, and Figure 1 details the read-path and update/invalidation data flows. GeoServer exposes WFS/WMS endpoints backed by PostGIS, Redis is used for per-cell caching on the read path, and Kafka/CDC events are used for spatially scoped invalidation on updates. This subsection focuses on system context and assumptions; middleware structure and request-handling behavior are specified in Section 3.3.

Layers are published in `EPSG:4326` (unless a request specifies another `srsName`/SRID) and responses are encoded as GeoJSON. Queries target urban layers with realistic skew, mixing small viewports (city blocks) with medium

windows (districts). Cache entries are keyed per H3 cell using a composite key schema (defined in Section 3.3.3); each value is stored with a per-layer TTL and may also be invalidated by change events. Redis is configured with bounded memory and an eviction policy [36].

Kafka provides in-partition ordering [37]; consumers are in a single group per environment so each partition is processed by exactly one consumer. Event handling is repeat-safe so duplicate invalidations are harmless.

A stable network is assumed during testing, and no background load is assumed except the experiment. GeoServer/PostGIS schemas or indexes are not changed; the middleware is non-invasive and standards-compliant on the wire.

3.3 Middleware Design and Implementation

This subsection specifies the middleware behavior in a reproducible way. Figure 5 defines the main middleware modules and responsibilities, and Figure 2 specifies the request-handling algorithm step-by-step. Deployment details (containers, versions, and hardware) are described in Section 3.5.1.

3.3.1 End-to-End Request Flow and Component Responsibilities

Figure 2 summarizes the read-path algorithm. For clarity and repeatability, the middleware handles each incoming WFS `GetFeature` request as follows:

1. Parse and validate required parameters (`service`, `request`, `typeNames`, and spatial filter); build an internal `QueryRequest`.
2. Extract the spatial footprint (BBOX or polygon) and the attribute filter (if present).
3. Normalize the attribute filter (Section 3.3.3) and compute `filterHash`.
4. Map the footprint to an H3 cell set at resolution r (Section 3.3.2).
5. Update per-cell hotness counters (exponential decay) and derive the cache-eligible subset E of cells for this request. If $E = \emptyset$, bypass the cache and forward the request to GeoServer (counted as a miss/bypass for metrics); otherwise continue with E.
6. Build per-cell Redis keys for E and issue a single `MGET` for all keys.
7. Classify the request over E as a *full hit*, *partial hit*, or *miss*:
   - **Full hit:** all entries for cells in E are present; proceed to composition.
   - **Partial hit/miss:** for each missing cell in E, forward a WFS `GetFeature` request to GeoServer (which queries PostGIS) and store the returned per-cell result in Redis with TTL.

   Cells not in E are always fetched from GeoServer for this request and are not admitted to Redis.
8. Compose the response by merging per-cell payloads (from cache and/or origin), deduplicating features, and reapplying the exact spatial and attribute filters (Section 3.3.4).
9. Return a GeoJSON `FeatureCollection` and record metrics (latency, keys-per-request, merge time, and hit class).

3.3.2 H3 Footprint Mapping and Request Bucketing

H3's polygon-to-cells ("polygonToCells") functions are used to map a request footprint to cell IDs [21]. A BBOX is converted into a polygon using its four corners in request coordinate order, and user polygons are used directly (including holes when present). For each request, the mapper returns the set of H3 cell IDs at resolution r, which become the unit of reuse and invalidation. The default polygonToCells rule includes a cell if its centroid lies within the footprint; to avoid boundary artefacts at coarser r, the composed response is trimmed by reapplying the exact request filters (Section 3.3.4).

Resolution controls granularity. Higher r yields smaller cells and more precise coverage but increases keys-per-request and composition overhead; lower r reduces key churn but may over-cover the footprint near boundaries. The main experiments compare $r \in \{7, 8, 9\}$.

The cache key includes the cell ID and a stable hash of the attribute filter (normalised ECQL). This keeps semantically different queries isolated while enabling reuse when both footprint and filter match.

On composed responses the original spatial filter is reapplied to remove any features picked up by boundary cells. Multi-polygons are supported by passing outer and inner rings to the polyfill. All cell operations are deterministic so repeated queries map to the same key set.

3.3.3 Caching in Redis: Key Schema and Result Storage

The cache entries are stored in Redis using a per-cell composite key: `<layer>:<srid>:<res>:<cellId>:<filterHash>:<version>`. Each request that spans multiple H3 cells triggers a single `MGET` for those keys to reduce round trips. `cellId` is the H3 64-bit identifier. `filterHash` is computed as `SHA-256` over a normalized attribute-filter string: URL-decoded, whitespace-collapsed, and with a canonical empty value `<no-filter>` when no attribute filter is present. `version` is a deployment-controlled epoch used for bulk invalidation (e.g., schema/data epoch), so changing it invalidates all existing keys without scanning.

The value layout is simple: a small header (timestamps, byte length, schema/version) and a payload. For vector responses GeoJSON Features or small, per-cell arrays of feature rows are stored. TTL is set at write and attached per key, Redis removes expired keys automatically. Memory pressure is controlled by `maxmemory` and an eviction policy (e.g., `allkeys-lru` or `allkeys-lfu`) so the cache behaves predictably when full.

3.3.4 Response Composition and Correctness Safeguards

The aggregator merges per-cell payloads into one GeoJSON `FeatureCollection`. To make composition reproducible, de-duplication uses a stable per-feature identifier with the following precedence: (i) GeoJSON `id` if present, else (ii) a configured per-layer primary-key property (e.g., `properties.<pk>`), else (iii) a deterministic fallback `SHA-256` over canonicalized geometry+properties. If none of these identifiers can be produced (unexpected/invalid payload), the middleware falls back to the miss path for correctness.

After merging, the middleware reapplies the *exact* request filters to the composed set: the same attribute predicate used to compute `filterHash` and the same spatial predicate implied by the request footprint (e.g., BBOX or

polygon). Features failing either predicate are removed before returning, which trims over-coverage introduced by cell boundaries. If a request contains an unsupported/unknown predicate that cannot be evaluated safely in the middleware, the request is treated as a miss and forwarded to GeoServer.

Partial hits are handled by fetching all missing cells from GeoServer before composing. Metrics are tagged by hit class (full hit/partial hit/miss) and include latency, keys-per-request, and merge time to support verification and debugging.

3.4 Freshness Mechanisms

TTL-based freshness is combined with event-driven invalidation. TTL gives a clear freshness lifetime (fresh vs. stale) that is easy to reason about and measure. Event-driven invalidation reacts to actual data changes and targets only the overlapping H3 cells, keeping hot regions up to date without broad purges.

3.4.1 TTL Policy and Configuration

On write, each key gets a TTL, keys expire automatically once the countdown reaches zero. TTL is set at insert time. Remaining lifetime is tracked with `TTL`/`PTTL` during debugging. TTLs are per layer: fast-changing layers get short TTLs, static base layers get longer ones.

Operationally, Redis removes expired keys and can also evict keys under memory pressure according to `maxmemory-policy`. An *allkeys* policy (LRU/LFU) is used when the dataset is entirely cacheable and memory-bounded, or a *volatile* policy is used when only TTL-tagged entries should be evicted. Metrics for expiry and eviction are also exposed so freshness decisions (TTL) can be separated from memory backpressure effects (evictions) in the results.

3.4.2 Kafka/CDC Event Processing and Spatial Invalidation

To cut staleness after updates, change events are consumed from PostgreSQL. The PostgreSQL connector streams inserts/updates/deletes from the WAL as structured messages; the middleware extracts the changed geometry, maps it to H3 cells, and invalidates the matching keys. This targets only the affected spatial scope and avoids global cache clears.

Kafka's ordering within a partition is relied on: by partitioning events on a stable key (e.g., feature ID), all changes for the same feature arrive in order. Delivery is at-least-once by default [38], so the invalidation handler is *idempotent* and *version-aware* (drops events that have already been processed at an equal or newer source LSN). Where a stricter pipeline is needed, Kafka's idempotent producers and transactions can be used to enable exactly-once processing, with added operational cost. These guarantees keep invalidation consistent even during retries or restarts.

For spatial targeting, the middleware takes the changed geometry from the CDC event ("after" geometry for inserts/updates; "before" geometry for deletes when available), maps it to H3 cell IDs at the cached resolution(s), and invalidates all cached entries for each affected cell. Because `filterHash` varies by query, invalidation is performed by deleting all keys that share the prefix `<layer>:<srid>:<res>:<cellId>:` (i.e., any `filterHash` under that cell), using a `SCAN MATCH` + batched `DEL` strategy. This ties freshness directly

to spatial overlap while ensuring that any cached query result for the affected cell is removed.

3.5 Experimental Methodology

The experiments compare the baseline stack against the cached stack under controlled, repeatable conditions using the run procedure in Figure 3. Each run follows the same stages (start containers → health checks → warm-up → steady-state measurement window → export metrics → reset state). Image tags/digests are pinned, and the request list and random seeds are fixed so the request mix is identical between configurations. Between runs, containers are restarted to clear Redis/PostGIS/GeoServer state and avoid cross-run contamination.

Measurements are taken only during the steady-state measurement window after warm-up (Figure 3). Collected metrics include latency (P50/P95/P99), achieved throughput, hit class rates (full/partial/miss), and backend resource usage (PostGIS CPU, GeoServer CPU/memory, Redis memory/evictions). External factors are kept stable (hardware, container limits, dataset, versions, and request list), so results are interpreted comparatively as deltas between configurations rather than universal absolute numbers.

3.5.1 Testbed and Deployment (Containers, Versions, Hardware)

All components run in Docker on a single host laptop. Images are pinned by tag and the exact image digest is recorded in the repository. The stack includes: GeoServer (WFS/WMS), PostgreSQL, PostGIS, Redis, the middleware proxy, Kafka, Prometheus, Grafana, Loki, and Promtail. Each service is launched via a compose file. The dataset is loaded once at startup, and containers are restarted between configurations to reset caches.

Hardware was a single laptop running Arch Linux (Lenovo Legion 7 15IMH05) with an Intel Core i7-10750H (6 cores / 12 threads) and 16 GiB RAM. The experimental stack ran in Docker containers (PostGIS, GeoServer, Redis, Kafka, and monitoring). Redis was configured with a fixed memory budget using `maxmemory`=256 MiB and the `allkeys-lru` eviction policy to bound cache growth. Prometheus scraped service metrics and Grafana dashboards summarized latency distributions, throughput, error rates, and Redis memory/evictions; log-based observability was provided via Loki/Promtail.

3.5.2 Baselines and Compared Configurations

Configurations are defined by the factor grid {cache on/off} × {Zipf_s (skew)} × {offered load (RPS)} × {H3 resolution r}. The main comparisons in this thesis evaluate baseline (cache off) versus cache on with $r \in \{7, 8, 9\}$ across the reported Zipf_s settings. For each configuration, the same dataset, request list (fixed seed), and container limits are used. Reported outcomes include end-to-end latency (P50/P95/P99), achieved throughput, backend resource usage, Redis memory/evictions, and (where enabled) freshness-related indicators.

3.5.3 Experiment Procedure

Each trial proceeds as follows:

1. Start all containers and wait for health checks to pass.
2. Warm up for 1 minute (not measured) using the same request list as the measurement phase.
3. Run the load generator at a constant arrival rate for a 4-minute steady-state measurement window, collecting metrics during this period.
4. Stop the load generator and export metrics from Prometheus and middleware logs for analysis.
5. Restart containers to reset state (Redis contents, database buffers, and service caches) before the next run/configuration.

This procedure corresponds to Figure 3. For each configuration, the request list and random seed are held fixed, and the configuration is repeated n times (e.g., n=5) to account for variability; medians and percentiles are then aggregated across repetitions.

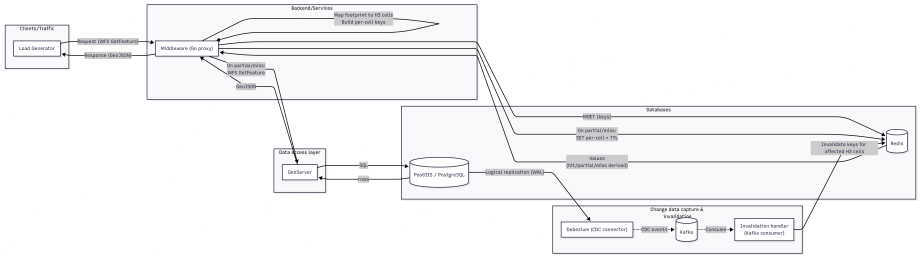


Figure 1: System architecture and data flows for the read path (WFS GetFeature) and the update/invalidation path.

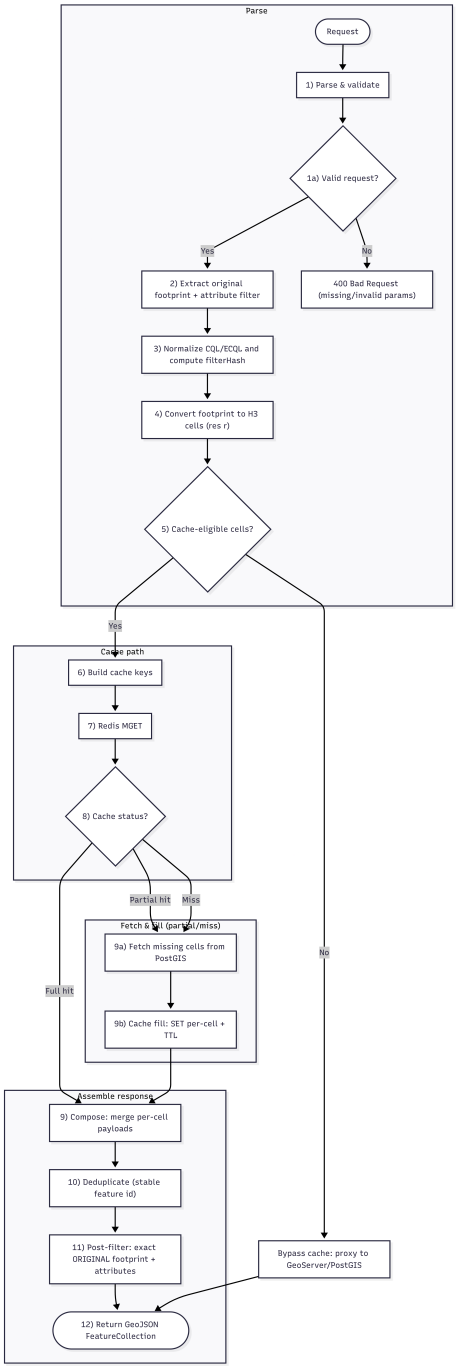


Figure 2: Step-by-step request handling in the middleware (parse → H3 mapping → Redis lookup → hit/miss handling → compose/dedup/filter → return).

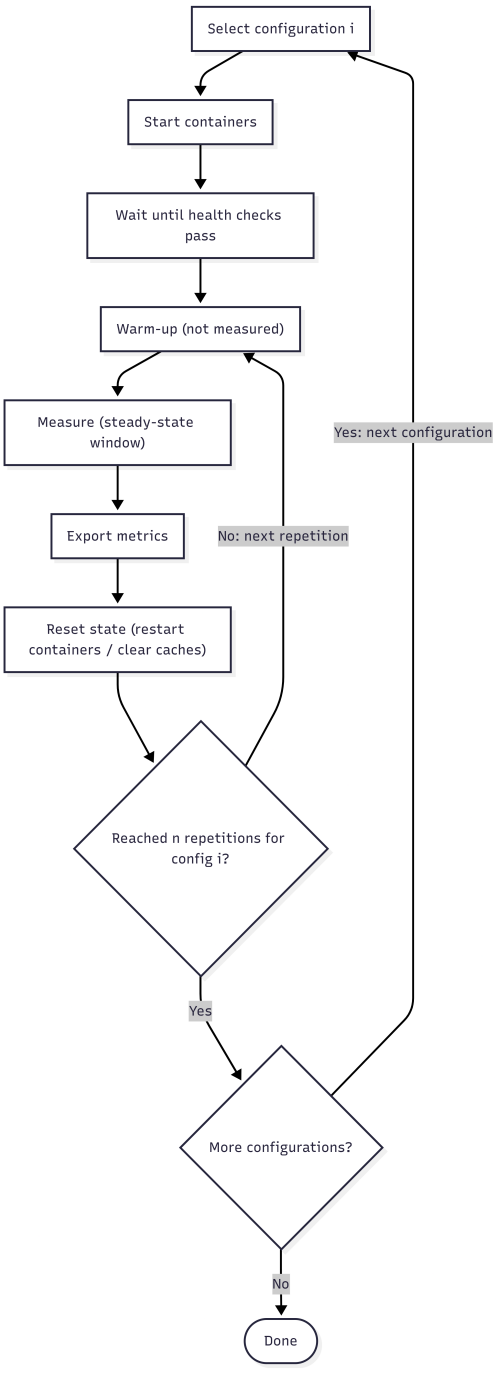


Figure 3: Experimental run procedure per configuration: warm-up (not measured), steady-state measurement window, export of metrics, reset, and repetition across configurations.

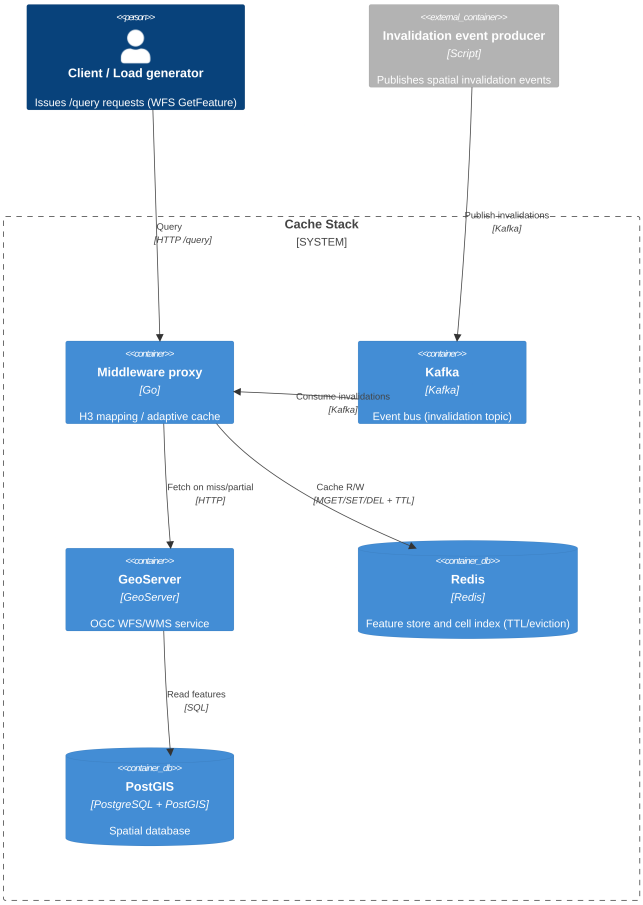


Figure 4: C4 container view of the docker-compose testbed: client queries the middleware; the middleware fetches from GeoServer/PostGIS on misses, caches in Redis, and consumes invalidation events from Kafka produced by an external publisher.

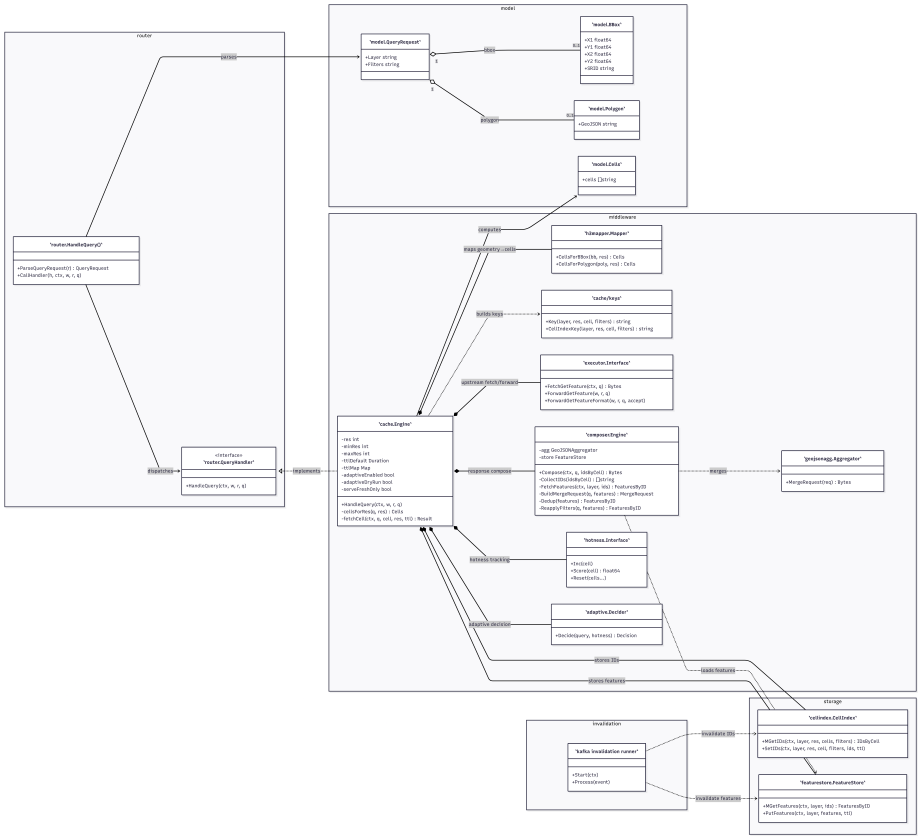


Figure 5: Middleware class/module diagram showing the request entrypoint (`router.HandleQuery`), cache engine (`cache.Engine`), H3 mapping, Redis-backed cell index and feature store, composition pipeline, adaptive decision components, and the Kafka invalidation runner.

4 Verification and Validation

This chapter verifies the experimental setup and supports the validity of the measured performance differences between the middleware cache and the baseline service. Validation is based on end-to-end latency percentiles (P50/P95/P99), throughput and error rates (to ensure offered load is actually sustained), and backend CPU/memory measurements for GeoServer and PostGIS.

4.1 Experimental Setup

All test conditions are kept fixed across configurations and only vary the parameters listed in the matrices (Table 2). The offered load is held constant per scenario, runs follow the same warm-up and timed window procedure, and the dataset, containers, and seeds are identical between configurations. Median values are computed across repeated runs per configuration. The number of repetitions and achieved throughput per run are listed in the overview table (Table 3).

Table 2: Experiment parameter matrix used across configurations.

Parameter	**Value**
BBOX samples	1024
Concurrency	32
Warm-up	30 s
Measurement window	60 s
Target RPS	600, 800 and 1000
Repetitions (n_{reps})	600: 1; 800: 3; 1000: 1
Scenarios	Baseline; Cache
H3 resolution (cache)	7, 8 and 9
TTL	60 s
Zipf s	1.1, 1.2, 1.3 and 1.4
Zipf v	1.0
Layer	`demo:NR_polygon`
Seed	123 456 789

Table 3: Overview of the 800 RPS scenarios: latency percentiles, throughput/errors, repetitions (n_{reps}), and backend resource usage (medians across repetitions; CPU is total across cores; memory in MiB).

			Latency (ms)			Run stats			Backend			
Scenario	r	Zipf s	P50	P95	P99	Thrpt (RPS)	Err	n_{reps}	PG CPU (%)	GS CPU (%)	PG Mem (MiB)	GS Mem (MiB)
Baseline	0	1.1	5.87	34.83	100.11	789.1	23	3	59.0	310.8	81	1020
Cache	7	1.1	1.08	6.42	13.49	800.0	5	3	6.1	26.3	84	1267
Cache	8	1.1	4.53	43.43	122.77	788.7	15	3	21.3	87.6	86	1301
Cache	9	1.1	56.90	380.59	972.75	259.9	328	3	54.4	195.3	86	1488
Baseline	0	1.2	5.58	30.77	95.31	787.9	7	3	56.6	300.4	82	1114
Cache	7	1.2	1.02	5.94	12.15	800.0	4	3	5.6	24.7	86	1476
Cache	8	1.2	3.67	33.99	104.12	789.2	53	3	16.8	67.3	86	1476
Cache	9	1.2	53.19	324.18	757.37	303.2	266	3	49.4	168.1	87	1518
Baseline	0	1.3	5.33	29.78	85.55	789.2	5	3	57.6	285.8	82	1132
Cache	7	1.3	0.97	5.31	9.91	800.0	4	3	4.6	19.9	85	1643
Cache	8	1.3	2.74	17.35	35.74	800.0	4	3	12.1	48.7	85	1657
Cache	9	1.3	52.69	280.44	721.87	326.4	152	3	42.3	134.6	85	1700
Baseline	0	1.4	5.10	26.95	90.33	789.5	21	3	56.0	282.7	81	1238
Cache	7	1.4	0.94	4.66	8.48	800.0	3	3	4.4	15.1	84	1690
Cache	8	1.4	2.33	13.37	27.34	800.0	0	3	9.3	33.9	83	1693
Cache	9	1.4	50.00	243.44	585.52	377.9	89	3	36.2	120.6	84	1713

4.2 Latency Results

Latency is reported as P50/P95/P99 for the end-to-end path, using medians across repetitions per configuration. Baseline (no cache) is compared with H3 cache at $r \in \{7, 8, 9\}$, focusing on the 800 RPS scenarios unless noted. Because cache hit ratio is not logged directly, reuse is interpreted indirectly via latency speedup (baseline/cache; Figure 9) and backend offload (reduced PostGIS CPU; Figure 10).

4.2.1 Latency vs Zipf skew

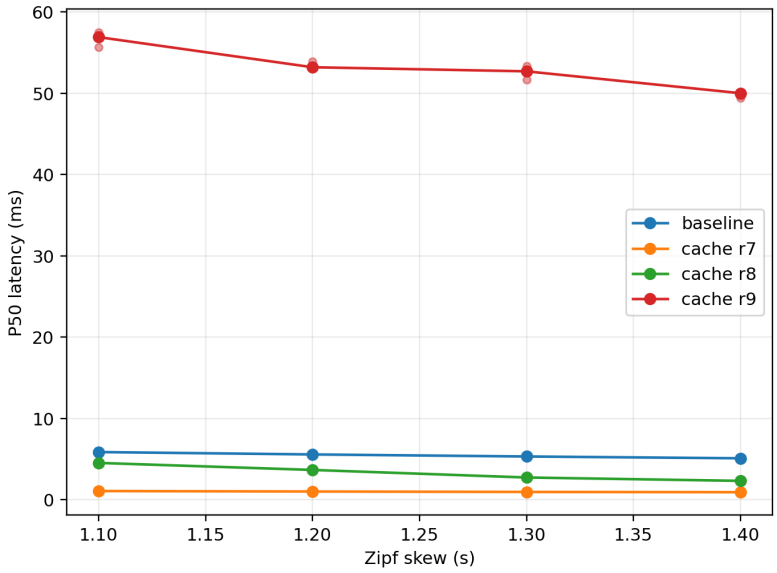
Figure 6 summarizes how skew (Zipf_s) changes latency at 800 RPS. As skew increases (from 1.1 to 1.4), both baseline and cache improve, but the cache at r=8 improves the most with skew, while r=7 remains consistently low and is best among the tested resolutions. Improved performance under higher skew is expected for coarser grids, which increase spatial reuse by grouping a larger area into each cache bucket. At Zipf_s =1.1 the baseline has P50 ≈5.87 ms, P95 ≈34.83 ms and P99 ≈100.11 ms, while r=7 has P50 ≈1.08 ms, P95 ≈6.42 ms and P99 ≈13.49 ms (∼5.4×, ∼5.4× and ∼7.4× lower; Figure 6). The same effect is shown as explicit speedup factors (baseline/cache) in Figure 9. At Zipf_s =1.4 the baseline P50/P95/P99 are ≈5.10/26.95/90.33 ms, and r=7 is ≈0.94/4.66/8.48 ms (∼5.4×, ∼5.8× and ∼10.6× lower; Figure 6). The r=8 cache improves with skew (e.g., P95 ≈43.43 ms at 1.1 down to ≈13.37 ms at 1.4; Figure 6b). Under low skew it can be slower than baseline in the tails, but it outperforms baseline once skew is higher ($Zipf_s \geq 1.3$) and remains better at P50 even at lower skews. The r=9 cache shows very high tail latency at 800 RPS (Figure 6c); the numbers for r=9 are interpreted with caution given its poor load sustain (Figure 8a and Figure 8b).

4.2.2 Latency vs H3 resolution

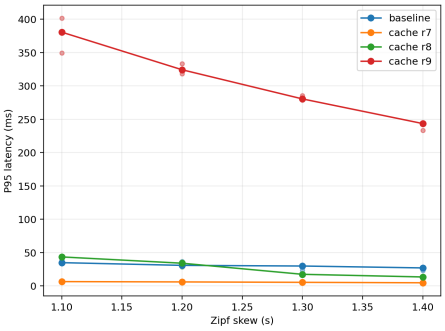
At 800 RPS, resolution matters (Figure 7). The cache at r=7 consistently gives the best latency across skews. However, this should be interpreted as a granularity–reuse trade-off: lower resolutions cover larger areas and therefore tend to improve reuse/hit probability, but they may reduce spatial selectivity (and can increase overfetch) when client requests are much smaller than a cell: at Zipf_s =1.1 it brings P50/P95/P99 to about 1.08/6.42/13.49 ms, and at 1.4 to about 0.94/4.66/8.48 ms (Figure 7). Compared to baseline at the same skews, this is roughly 5×–6× lower P50/P95 and 7×–11× lower P99. The r=8 cache is mixed: under low skew (1.1–1.2) its tails (P95/P99) are above baseline, but under higher skew (1.3–1.4) it improves and beats baseline, but it still beats the baseline in all skews for P50. The r=9 cache performs poorly at 800 RPS (very large P95/P99) and does not sustain the offered load (Figure 8a and Figure 8b); its percentiles are not directly comparable and focus on r=7/8 vs. baseline for the main conclusions.

4.2.3 Throughput, errors, and stability

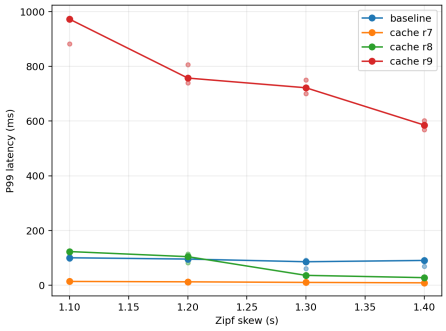
It is checked whether each configuration actually sustains the offered 800 RPS (Figure 8). Baseline is close to target across skews (throughput median ≈789.1 RPS, i.e., ∼0.985–0.987 of target) with low error counts (Figure 8b).



(a) P50.

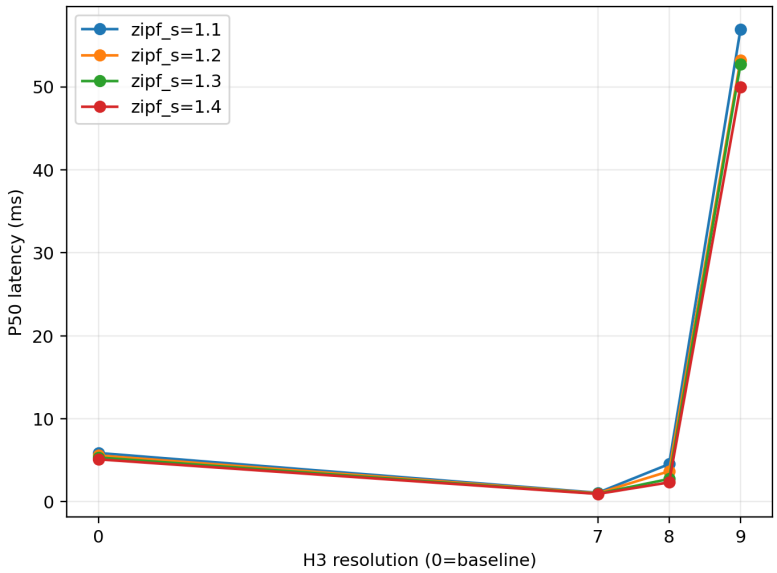


(b) P95.

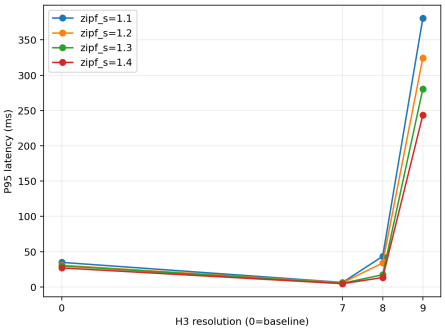


(c) P99.

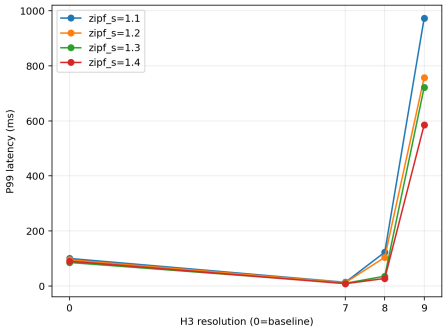
Figure 6: End-to-end latency vs. Zipf skew at 800 RPS (medians across repetitions).



(a) P50.



(b) P95.



(c) P99.

Figure 7: Latency vs. H3 resolution at 800 RPS, grouped by Zipf skew (medians across repetitions).

The r=7 cache meets target exactly (800.0 RPS median across skews) and has very few errors (Figure 8a and Figure 8b). The r=8 cache is split: at low skew it sits near baseline (throughput ∼0.986-0.987 of target with some errors; Figure 8a and Figure 8b), while at higher skew (1.3–1.4) it hits the target cleanly with zero or near-zero errors (Figure 8b). The r=9 cache does *not* sustain load (only ∼0.33–0.47 of target, with many errors; Figure 8a and Figure 8b), so its latency numbers are not directly comparable. These checks support that the baseline, r=7, and (under higher skew) r=8 comparisons are valid for latency.

4.3 Backend Load

This section focuses on resource impact at 800 RPS (Figure 10). The clearest effect is on PostGIS CPU (Figure 10a). Across Zipf_s $\in \{1.1, 1.2, 1.3, 1.4\}$, the r=7 cache reduces median PostGIS CPU by about one order of magnitude (e.g., ∼59.0%→6.1% at 1.1; ∼56.0%→4.4% at 1.4; Figure 10a). r=8 also offloads the database, but less so at low skew (e.g., ∼59.0%→21.3% at 1.1), improving as skew increases (down to ∼9.3% at 1.4; Figure 10a). GeoServer CPU follows the same pattern (Figure 10b): baseline medians of ∼282.7–310.8% drop to ∼15–26% for r=7 and to ∼34–88% for r=8, depending on skew.

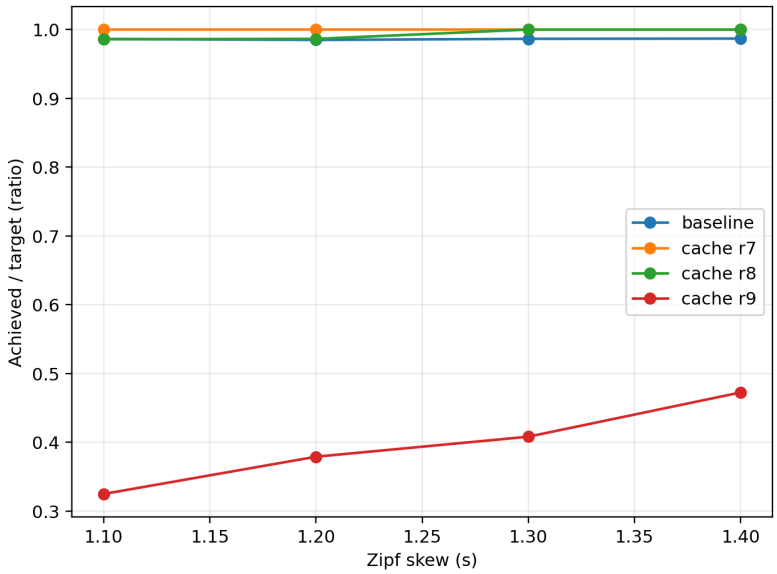
Memory is more nuanced (Figure 11a and Figure 11b). PostGIS memory changes little (Figure 11a), but GeoServer memory rises under caching (e.g., ∼1.02–1.24 GiB baseline vs. ∼1.27–1.69 GiB for r=7 across skews; Figure 11b). This cost is the main trade-off for the CPU savings and lower latency. Finally, throughput viability supports these comparisons (Figure 8a and Figure 8b): baseline sustains ∼0.985 of target on median; r=7 is effectively at target (∼0.99998); r=8 ranges from near-baseline at low skew to on-target at higher skew. The r=9 cache is not viable at 800 RPS (under-delivers with many errors; Figure 8a and Figure 8b), so it is excluded from backend comparisons at this load.

4.4 Load Sensitivity: 600 vs 800 vs 1000 RPS

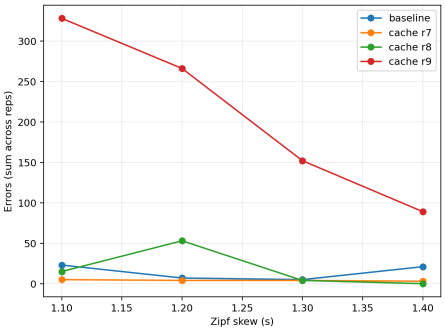
To examine how effects scale with load, Zipf_s=1.3 is fixed and baseline vs. cache r=8 are compared at 600/800/1000 RPS (Figures 12a–13a and Table 4). At 600 RPS, baseline P50/P95/P99 are ∼5.16/25.8/75.7 ms; r=8 lowers them to ∼2.57/16.3/32.4 ms (about 2.0×–1.6×–2.3× faster; Figure 12b and Figure 12c), and PostGIS CPU drops from ∼44.5% to ∼8.9% (Figure 13b). Both configurations meet target throughput (∼1.00; Figure 13a).

At 800 RPS, the gap widens: baseline P50/P95/P99 ∼5.33/29.8/85.5 ms vs. r=8 at ∼2.74/17.4/35.7 ms (∼ 1.9×—1.7×—2.4× faster; Figure 12b and Figure 12c), while PostGIS CPU falls from ∼57.6% to ∼12.1% (Figure 13b). The achieved/target ratio improves from ∼0.986 (baseline) to ∼0.99997 (cache; Figure 13a), indicating better headroom.

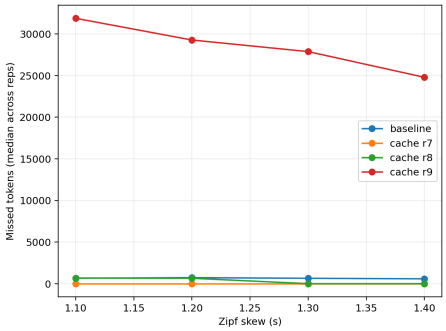
At 1000 RPS, baseline starts to strain: P50/P95/P99 rise to ∼21.33/67.6/111.3 ms and achieved/target dips to ∼0.971 (Figure 12b, Figure 12c, and Figure 13a). The r=8 cache remains more stable: P50/P95/P99 ∼4.09/39.2/98.5 ms and achieved/target ∼0.980 (Figure 12b, Figure 12c, and Figure 13a). PostGIS CPU falls from ∼83.2% (baseline) to ∼15.0% (cache; Figure 13b). In short, as offered load increases, the cache keeps tails in check and preserves throughput viability (Figure 12b, Figure 12c, and Figure 13a). Because r=7 already dominates at 800 RPS, r=8 is used here to show the "middle" case: even when r=8



(a) Achieved throughput / target.

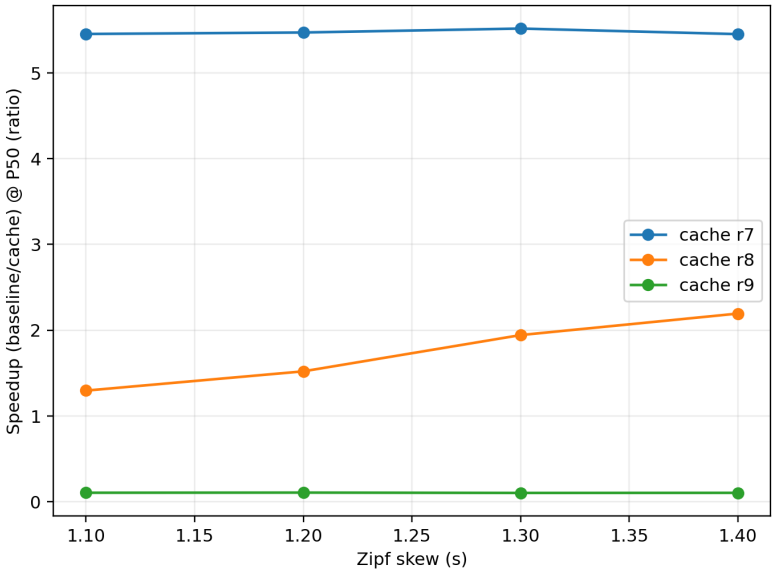


(b) Errors.

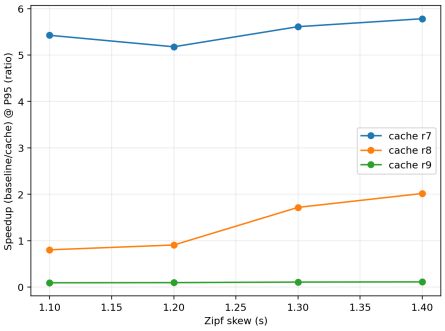


(c) Missed tokens.

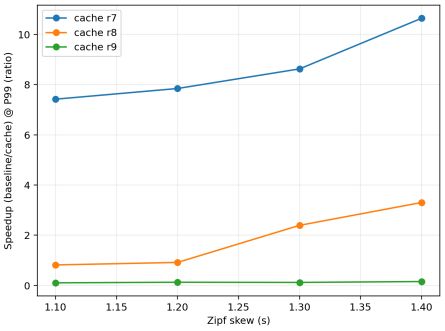
Figure 8: Throughput and stability indicators vs. Zipf skew at 800 RPS (medians across repetitions).



(a) P50.

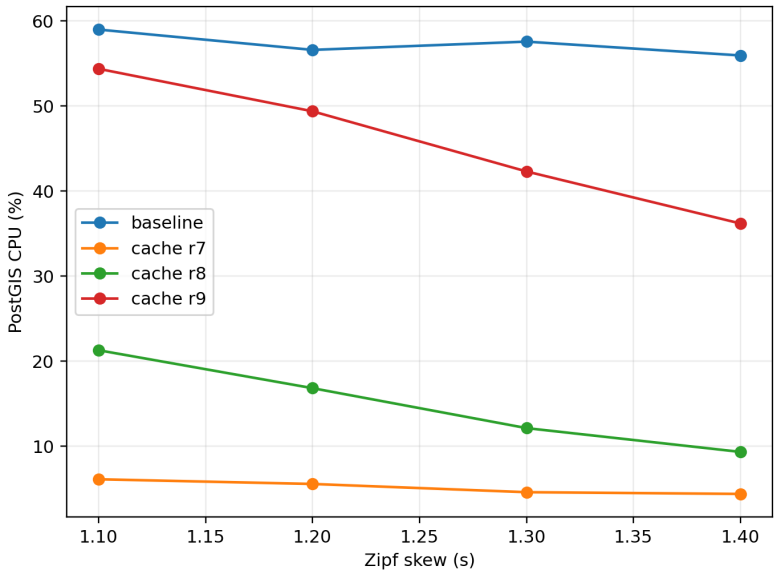


(b) P95.

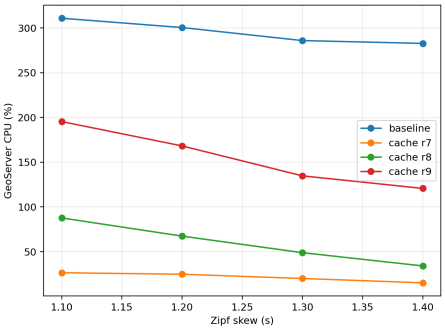


(c) P99.

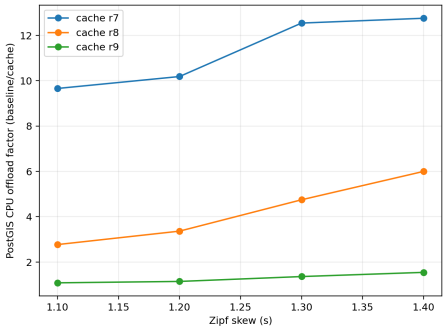
Figure 9: Latency speedup (baseline/cache) vs. Zipf skew at 800 RPS.



(a) PostGIS CPU utilization.

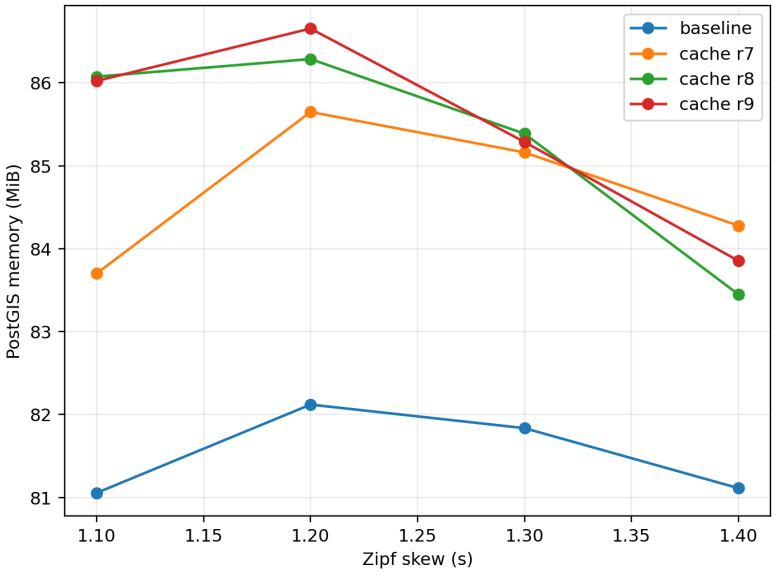


(b) GeoServer CPU utilization.

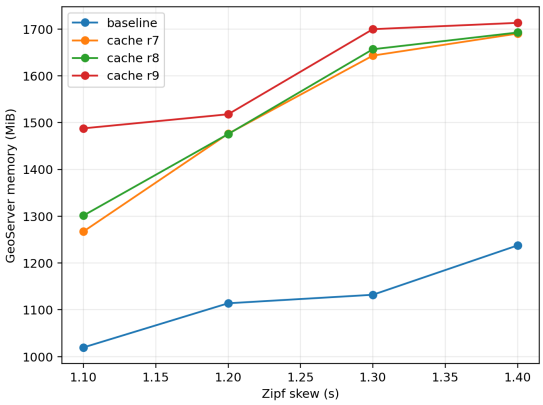


(c) PostGIS CPU offload factor (baseline/cache).

Figure 10: Backend CPU load and inferred database offload vs. Zipf skew at 800 RPS (medians across repetitions).



(a) PostGIS memory usage.



(b) GeoServer memory usage.

Figure 11: Backend memory usage vs. Zipf skew at 800 RPS (medians across repetitions).

Table 4: Load sensitivity summary for Zipf_s=1.3 comparing baseline vs. cache (r=8) across 600/800/1000 RPS.

Scenario	r	Target (RPS)	P50 (ms)	P95 (ms)	P99 (ms)	Achieved (RPS)	Err	PG CPU (%)	GS CPU (%)
Baseline	0	600	5.16	25.80	75.73	598.8	1	44.5	217.0
Cache	8	600	2.57	16.32	32.44	600.0	2	8.9	34.7
Baseline	0	800	5.33	29.78	85.55	789.2	5	57.6	285.8
Cache	8	800	2.74	17.35	35.74	800.0	4	12.1	48.7
Baseline	0	1000	21.33	67.56	111.33	970.9	5	83.2	392.7
Cache	8	1000	4.09	39.22	98.47	979.7	2	15.0	56.4

Table 5: Best cache resolution r per Zipf skew at 800 RPS based on latency percentiles.

Zipf s	Best r (P50)	Best P50 (ms)	Best r (P95)	Best P95 (ms)	Best r (P99)	Best P99 (ms)
1.1	7	1.08	7	6.42	7	13.49
1.2	7	1.02	7	5.94	7	12.15
1.3	7	0.97	7	5.31	7	9.91
1.4	7	0.94	7	4.66	7	8.48

is not the absolute best at low skew, it still delivers growing benefits as load climbs.

4.5 Verification and validation summary

This chapter verifies that the reported performance differences are measured under comparable and stable conditions, and validates that the observed effects are interpretable.

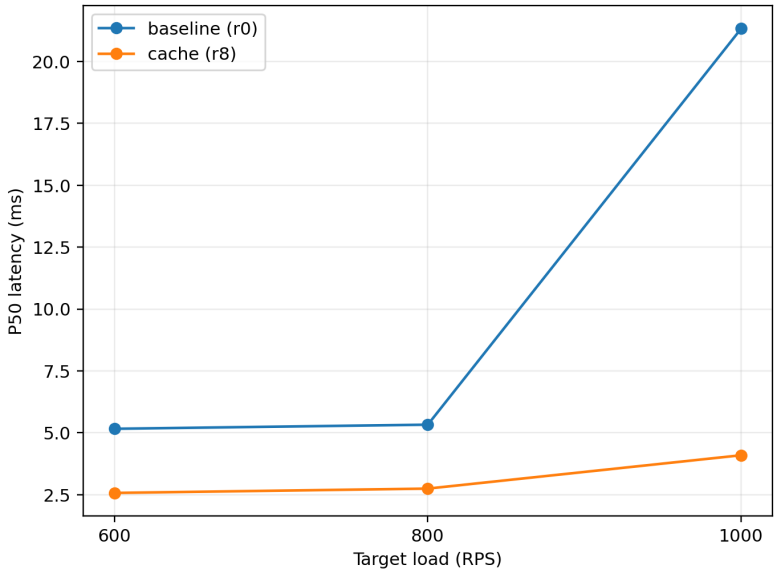
Controlled conditions: All configurations are executed with the same dataset, fixed seeds, pinned container images, identical warm-up and measurement windows, and only the parameters in Table 2 are varied. Reported metrics are medians across repetitions, with repetitions and achieved throughput per run listed in Table 3.

Offered-load viability: Comparisons are emphasized only when the system sustains the offered load with acceptable error rates. Baseline, r=7, and (for moderate/high skew) r=8 maintain throughput close to the 800 RPS target, while r=9 under-delivers and produces many errors at 800 RPS, so its latency percentiles are not treated as comparable at that load (Figure 8).

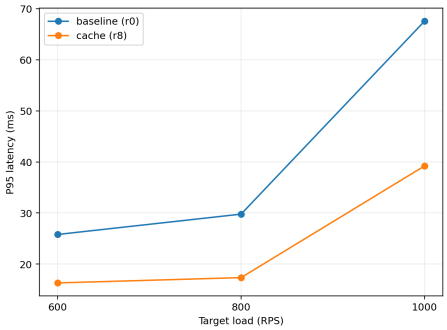
Interpretability of effects: Latency changes are interpreted together with backend CPU/memory to ensure improvements correspond to reduced backend work rather than measurement artifacts or instability. The load sweep at Zipf s=1.3 further checks that observed behavior is consistent as offered load increases (Table 4).

4.6 Results summary

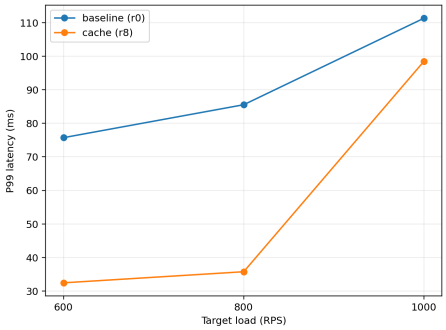
Across the 800 RPS experiments (Zipf $s \in \{1.1, 1.2, 1.3, 1.4\}$), the middleware cache delivers large improvements in end-to-end latency and substantially reduces database work. The strongest configuration on the testbed is H3 resolution r=7. Compared to the baseline (no cache), r=7 reduces median latency (P50) from roughly 5.1–5.9 ms to about 0.94–1.08 ms, and tail latency (P95/P99) from roughly 27–35 ms / 90–100 ms to about 4.7–6.4 ms / 8.5–13.5 ms (Figures 6 and



(a) P50.

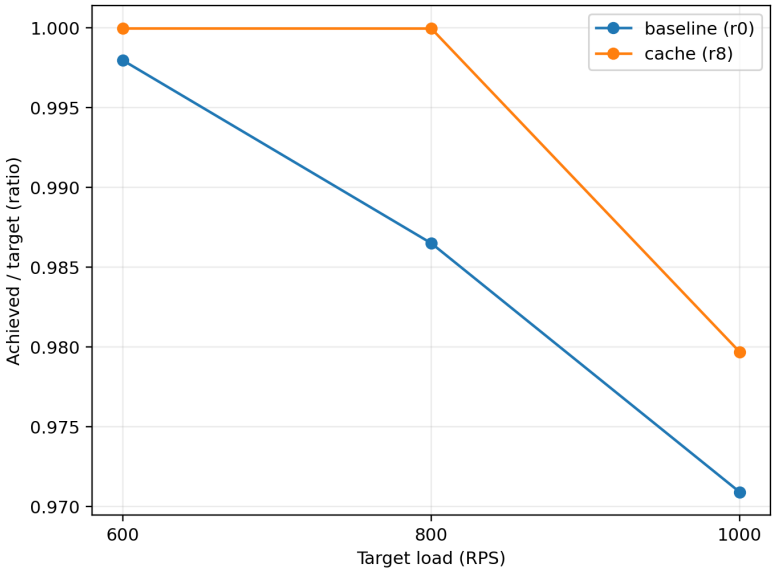


(b) P95.

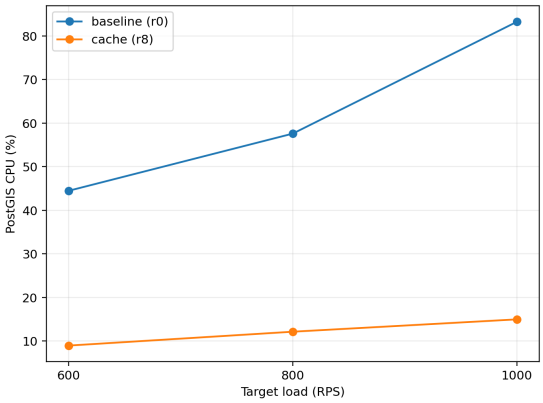


(c) P99.

Figure 12: Latency vs. offered load (RPS) at Zipf_s=1.3 for baseline vs. cache (r=8).



(a) Achieved throughput / target.



(b) PostGIS CPU utilization.

Figure 13: Throughput viability and backend load vs. offered load (RPS) at Zipf_s=1.3 for baseline vs. cache (r=8).

9). In parallel, PostGIS CPU is reduced by about an order of magnitude under the same offered load (e.g., $\sim$59% $\rightarrow$ 6% at s=1.1 and $\sim$56% $\rightarrow$ 4% at s=1.4; Figure 10a), indicating that the gains come from avoiding repeated database evaluation in hotspot regions, not only from faster response serialization.

Resolution affects the balance between reuse and per-request overhead. At r=8, performance is mixed at low skew (heavier tails than baseline at s=1.1–1.2) but improves clearly as skew increases ($s \geq 1.3$), where reuse is high enough to offset extra key and merge work (Figure 7). At r=9, the cache is not viable at 800 RPS on this testbed (throughput shortfall and many errors), so it is excluded from the main conclusions at this load (Figure 8).

The primary trade-off observed is memory on the GeoServer side: caching increases GeoServer memory by roughly 20–40% in the 800 RPS scenarios (Figure 11b), while PostGIS memory remains close to baseline. Overall, the results support that an H3-guided, Redis-backed middleware cache can substantially improve responsiveness and backend headroom for skewed vector workloads, with r=7 as the most robust starting point for the evaluated setup.

5 Result

In the work, the following results were produced:

1. Existing solutions for caching and freshness in GeoServer/PostGIS-style vector services were studied, and requirements were derived for hotspot-heavy workloads. The main finding is that common solutions work well for raster tiles or identical repeated requests, but they reuse poorly when vector queries only partially overlap, and freshness handling often becomes either coarse (invalidate too much) or slow (serve stale data too long).

2. Based on these requirements, an H3-keyed caching approach for vector results was designed, including cache keys, multi-cell composition, and correctness safeguards. The design enables reuse across overlapping requests by caching per H3 cell and then composing responses while reapplying the original spatial and attribute filters to keep results correct. In the evaluation, this approach reduced end-to-end latency and lowered PostGIS work under skew compared to the no-cache baseline.

3. A Go-based middleware prototype was implemented to realize the design, integrating Redis-backed per-cell caching and a deterministic compose/dedup/filter pipeline. The prototype supports different H3 resolutions (r) and logs the overhead signals that matter. In the tested workloads, r=7 was the most robust choice on this setup, while r=8 helped more as skew increased but had higher overhead at lower skew.

4. The prototype was experimentally evaluated in a containerized testbed under controlled skewed workloads. The evaluation compared configurations across latency percentiles, throughput stability, backend load, and memory footprint, and it also covered the effect of freshness choices. Combining per-key TTL with event-driven invalidation (Kafka-driven, spatially scoped) gives the best practical balance: TTL provides a clear upper bound on staleness, while targeted invalidation removes affected cells quickly without flushing unrelated hotspots.

6 Discussion

This chapter interprets the results in light of the research questions, first positioning the proposed middleware relative to existing caching approaches and their limitations, and then explaining *why* the cache helps under skew, what trade-offs arise from H3 resolution and composition overhead, and how freshness mechanisms affect performance. It is organised by research question first, followed by limitations and threats to validity, and sustainability.

6.1 Discussion by research question

6.1.1 RQ0: Existing solutions and gap

In GeoServer/PostGIS-style deployments, caching is typically strongest for raster tiles (fixed grids), while vector WFS queries are harder to cache because footprints and attribute predicates vary per request. Whole-request caching can help when identical viewports repeat, but it captures little reuse when requests only partially overlap, which is common during interactive panning/zooming. Database-side optimisations such as GiST indexing reduce the cost of each query, but they do not exploit cross-request overlap reuse: the database still repeats planning, index traversal, and exact geometry checks for hotspot regions.

The gap for hotspot-heavy, dynamic vector workloads is therefore a deployable, standards-compatible approach that reuses work across *partially overlapping* requests and keeps cached results fresh without broad purges. This thesis addresses that gap by using H3 cells as reusable spatial units for overlap reuse and by combining per-key TTL with event-driven invalidation so that only affected spatial regions are invalidated.

6.1.2 RQ1: Effectiveness under skewed workloads

Across the 800 RPS scenarios, the cache clearly improves responsiveness and reduces backend work when access is skewed. The strongest result is that r=7 consistently lowers P50/P95/P99 versus baseline for all skews that were tried (cf. Figures 6 and 7). The effect grows with skew: as Zipf_s increases, more requests overlap in a small set of cells, so reuse goes up and the database path is avoided more often. This matches what was observed in the PostGIS CPU plots, where r=7 cuts median CPU by roughly an order of magnitude at 800 RPS (Figure 10a). In other words, the middleware does not just make hits faster; it also keeps the miss path less congested by preventing queues from building up.

The tails matter most. The P95/P99 gaps (Figures 9 and 6) show that the cache reduces the slowest user-visible responses, not only the median. It was also checked that these gains are not an artefact of under-delivering load: r=7 meets the target throughput with low errors (Figure 8). By contrast, r=9 fails to sustain 800 RPS (missed tokens and errors), which explains its very high tails so its results are not directly comparable at this load.

Load sensitivity at Zipf_s=1.3 backs the same story. At 600/800/1000 RPS, the cache keeps tails lower and the achieved/target ratio closer to 1, while baseline starts to slip at 1000 RPS (Figures 12b–13a). Practically, this means caching buys headroom: for the same hardware, the system stays inside latency SLOs at higher offered load.

Takeaway. Under skew (which is common in maps), an H3-keyed cache in front of PostGIS delivers meaningful end-to-end wins: lower tail latency, higher achieved throughput, and large DB CPU offload. The effect is strongest when popular areas concentrate demand; it weakens, but does not disappear, as skew decreases.

6.1.3 RQ2: Granularity trade-offs (H3 resolution)

Resolution controls two things: spatial reuse vs. per-request overhead. Coarser grids (r=7 here) group more queries into the same buckets, which raises hit probability and reduces pressure on PostGIS. That shows up directly in the latency and CPU plots at 800 RPS, where r=7 dominates across skews (Figures 7 and 10a). The cost is lower spatial selectivity: each cell covers a larger area, so composition may bring in extra features near boundaries.

Finer grids (r=8 and especially r=9) increase specificity, which can help for very small viewports, but they also increase keys-per-request and merge work. That cost shows up in the tails and in throughput stability: r=8 is viable and becomes better as skew increases (more reuse despite the extra keys), but r=9 is over the edge at 800 RPS on the setup for this thesis (Figures 8a and 8b). Memory-wise, GeoServer RAM rises under caching (the main trade-off), but differences across r are smaller than the latency/CPU effects (Figure 11b).

Guidelines. Start with the coarsest resolution that still aligns with typical viewport sizes (here r=7), then move one step finer (r=8) only if clients routinely query very small areas or if over-coverage becomes a correctness/size issue. Watch two indicators when tuning: keys-per-request and merge time (composition overhead), and DB offload. If keys-per-request grows quickly or tails rise, prefer coarser cells or introduce an "umbrella" coarse entry for bursts, keeping only a subset of fine cells hot where there is clear reuse.

6.1.4 RQ3: Freshness and invalidation policy implications

TTL gives a simple and measurable upper bound on staleness, while event-driven invalidation cuts staleness sooner for changed regions. In practice, the two should be tuned together. A longer TTL improves hit ratio (and thus latency/CPU), but risks serving older data; shorter TTLs lower staleness, but they reduce reuse and can push more requests down the miss path. Event-driven invalidation helps square this circle by removing only the cells that overlap changed features, so popular but unchanged cells remain hot.

Operationally, two practices were useful. First, set TTLs per layer based on update cadence (short for frequently edited layers, longer for slowly changing base data) and monitor the fraction of responses served past the target. Second, keep invalidation idempotent and version-aware so duplicates or replays are harmless; this avoids broad purges and keeps the cache stable under churn. With these guardrails in place, the main effect of freshness tuning is a smooth trade-off: relaxing TTLs increases reuse and lowers latency until the staleness budget is hit; tightening them does the opposite.

6.2 Limitations and threats to validity

6.2.1 Synthetic workload vs. real traffic

The request mix and Zipf-like skew that are used in this thesis are representative but not exhaustive; real deployments may combine diverse layers or constant cycles that change hot regions over time. Results should be treated as comparative evidence (cache vs. baseline under the same conditions), not as universal absolutes across all catalogues and user behaviors.

6.2.2 Single-host, containerized testbed

Running all components on one machine (with Docker networking) can understate network variance that appears in multi-host clusters. Kernel scheduling and IO paths also differ between laptops and production servers.

6.2.3 Generalizability across schemas and predicates

The thesis focused on common spatial filters in WFS/WMS requests. Workloads with complex attribute joins, heavy server-side processing, or very small polygons may behave differently and benefit from other H3 resolutions. Using spatial indexes in PostGIS helps, but cannot remove all of these differences.

6.2.4 Freshness and delivery guarantees

TTL limits how long cached data can become stale, while event-driven invalidation reduces staleness when updates happen. In practice, CDC and messaging can be delayed or duplicated, so we rely on ordered partitions and idempotent handlers to maintain cache consistency. Systems that require strict transactional guarantees must account for the added complexity and overhead of Kafka's exactly-once mechanisms.

6.2.5 Cache memory and eviction artifacts

With limited Redis memory, eviction policy affects tail latency during churn. An *allkeys-LFU* or *allkeys-LRU* policy generally works best, but sudden access pattern changes or wide scans can still evict useful entries. Redis documentation on eviction and latency should be kept in mind when interpreting tail-latency spikes under memory pressure.

6.3 Sustainability (Energy and efficiency)

Caching popular regions reduces redundant database work, which lowers CPU time on misses and can reduce energy-per-request. The Green Software Foundation's SCI specification encourages tracking carbon intensity at the interface boundary (per-request), which aligns with measuring tail latency, offload factor, and achieved/target throughput to quantify efficiency gains from caching [39]. Where possible, schedule heavy backfills or pre-warming outside peak grid-carbon windows. Also, energy use scales with overprovisioning; so keep Redis and GeoServer memory allocations proportional to observed working sets and limit concurrency to avoid wasteful queueing.

7 Conclusions

7.1 Answers to the research questions

7.1.1 RQ0: Existing solutions and gap

Existing caching approaches in GeoServer/PostGIS-style vector services either focus on raster tiles, cache only identical full requests, or rely on database indexing and buffering. These options do not reliably capture reuse when queries overlap but are not identical, and freshness under updates often leads to coarse invalidation. The gap is thus a practical, standards-compatible solution that exploits overlap reuse at a stable spatial unit while keeping staleness bounded.

The H3-keyed, per-cell middleware cache proposed here fills this gap by enabling composition across overlapping BBOX/polygon queries and by pairing TTL with Kafka/CDC-driven spatial invalidation to invalidate only the affected H3 cells.

7.1.2 RQ1: Effectiveness

The H3-based cache improves responsiveness and reduces backend load under skewed access. At 800 RPS, the best configuration (r=7) consistently lowered P50/P95/P99 compared to baseline, with the largest gains in the tails. The effect strengthened as skew increased, because more requests overlapped in the same cells and were served from memory. This also showed up as a clear PostGIS CPU offload (roughly an order of magnitude in the median plots), and better achieved/target throughput at high load. The main cost was a moderate increase in GeoServer memory, which is acceptable given the latency and offload benefits.

7.1.3 RQ2: Granularity

Resolution controls reuse vs. overhead. Coarser cells (r=7) grouped nearby views and increased reuse, giving the best tail latency and the highest database offload in the runs. Finer cells (r=8) helped some small-window cases and improved as skew rose, but carried higher composition and key-churn costs; at 800 RPS, r=9 was not viable on the testbed. In practice, starting from a coarse resolution that roughly matches typical view sizes, and only stepping finer when needed, was the most robust strategy. Keys-per-request and merge time were the most useful live indicators when tuning.

7.1.4 RQ3: Freshness

TTL gives a clear age bound; event-driven invalidation then shortens the stale window where updates occur. Using per-layer TTLs matched to update cadence, plus idempotent, version-aware invalidation, kept popular cells hot while replacing only the regions that changed. The trade-off is smooth: longer TTLs raise reuse until a staleness budget is hit; shorter TTLs reduce reuse but tighten freshness. With this setup, the hybrid (TTL and events) offered good latency while keeping staleness bounded for edited areas.

7.2 Contributions

- **Design and prototype.** A standards-compatible middleware that maps WFS/WMS footprints to H3 cells, caches per-cell results in Redis, and composes responses on cache hits.
- **Freshness mechanism.** A hybrid scheme that combines per-key TTL with CDC/Kafka-driven spatial invalidation, implemented with idempotent, version-aware handlers.
- **Evaluation method.** A containerized, repeatable benchmark with fixed seeds and steady windows, reporting P50/P95/P99, backend CPU, memory, throughput viability, and cache stats.
- **Empirical findings.** Clear evidence that H3 based caching improves latency compared to baseline (no caching). r=7 dominated at 800 RPS across skews; r=8 improved with higher skew; r=9 was unstable on this hardware/load.
- **Practical guidance.** Rules of thumb for choosing resolution, watching keys-per-request and merge time, sizing memory, and setting per-layer TTLs.

7.3 Future work

- **Multi-resolution caching.** Use both coarse and fine H3 cells at the same time: coarse cells for short traffic bursts and fine cells for frequently accessed small areas, and choose which one to serve per request.
- **Cache admission and eviction.** Study simpler admission and replacement strategies that reduce cache churn during sudden workload changes, while still prioritising frequently accessed regions.
- **Whole-query caching.** Add optional cache entries for very common, identical bounding boxes or polygons to avoid splitting and recombining results on hot paths.
- **Staleness monitoring.** Record how old cached data is when served and how long invalidation takes, and use this information to automatically tune TTL values to meet latency goals.
- **Broader workloads.** Evaluate the approach on more complex queries, such as attribute joins, server-side transformations, and very small geometries, and test on multi-node deployments.
- **Operational robustness.** Measure recovery times after failures, study stronger delivery guarantees where needed, and analyse cost and energy usage for different cache sizes.
- **System integration.** Package the solution as a GeoServer extension or a transparent proxy, and provide simple configuration tools for per-layer defaults such as resolution, TTL, and invalidation scope.

References

[1] Open Geospatial Consortium. *Web Map Tile Service (WMTS) Standard*. https://www.ogc.org/standards/wmts/. 2025.

[2] Open Geospatial Consortium. *Web Feature Service (WFS) Standard*. https://www.ogc.org/standards/wfs/. 2025.

[3] Paul Ramsey, Mark Leslie, and PostGIS contributors. *Spatial Indexing: Introduction to PostGIS*. https://postgis.net/workshops/postgis-intro/indexing.html. 2012–2023.

[4] Christian Winter et al. "GeoBlocks: A Query-Cache Accelerated Data Structure for Spatial Aggregation over Polygons". en. In: *Proceedings of the 24th International Conference on Extending Database Technology (EDBT)*. OpenProceedings.org, 2021. DOI: 10.5441/002/EDBT.2021.16. URL: https://openproceedings.org/2021/conf/edbt/p78.pdf.

[5] Andreas Kipf et al. *Adaptive Main-Memory Indexing for High-Performance Point-Polygon Joins*. en. 2020. DOI: 10.5441/002/EDBT.2020.31. URL: https://openproceedings.org/2020/conf/edbt/paper_190.pdf.

[6] Lauro Lins, James T. Klosowski, and Carlos Scheidegger. "Nanocubes for Real-Time Exploration of Spatiotemporal Datasets". In: *IEEE Transactions on Visualization and Computer Graphics* 19.12 (Dec. 2013), pp. 2456–2465. ISSN: 1077-2626. DOI: 10.1109/tvcg.2013.179. URL: http://dx.doi.org/10.1109/TVCG.2013.179.

[7] Dimitris Papadias et al. "Efficient OLAP Operations in Spatial Data Warehouses". In: *Advances in Spatial and Temporal Databases*. Springer Berlin Heidelberg, 2001, pp. 443–459. ISBN: 9783540477242. DOI: 10.1007/3-540-47724-1_23. URL: http://dx.doi.org/10.1007/3-540-47724-1_23.

[8] Saptashwa Mitra et al. "STASH: Fast Hierarchical Aggregation Queries for Effective Visual Spatiotemporal Explorations". In: *2019 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, Sept. 2019, pp. 1–11. DOI: 10.1109/cluster.2019.8891029. URL: http://dx.doi.org/10.1109/CLUSTER.2019.8891029.

[9] Chang Liu, Brendan C. Fruin, and Hanan Samet. "SAC: semantic adaptive caching for spatial mobile applications". In: *Proceedings of the 21st ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. SIGSPATIAL'13. ACM, Nov. 2013, pp. 174–183. DOI: 10.1145/2525314.2525366. URL: http://dx.doi.org/10.1145/2525314.2525366.

[10] Yiwen Wang. "Vecstra: An Efficient and Scalable Geo-spatial In-Memory Cache". In: *Proceedings of the VLDB 2018 Ph.D. Workshop, August 27, 2018, Rio de Janeiro, Brazil*. CEUR Workshop Proceedings, 2018. URL: https://ceur-ws.org/Vol-2175/paper06.pdf.

[11] Stavros Maroulis et al. "Visualization-Aware Time Series Min-Max Caching with Error Bound Guarantees". In: *Proceedings of the VLDB Endowment* 17.8 (Apr. 2024), pp. 2091–2103. ISSN: 2150-8097. DOI: 10.14778/3659437.3659460. URL: http://dx.doi.org/10.14778/3659437.3659460.

[12] Jianliang Xu, Xueyan Tang, and Dik Lun Lee. “Performance analysis of location-dependent cache invalidation schemes for mobile environments”. In: *IEEE Transactions on Knowledge and Data Engineering* 15.2 (Mar. 2003), pp. 474–488. ISSN: 1041-4347. DOI: 10.1109/tkde.2003.1185846. URL: http://dx.doi.org/10.1109/TKDE.2003.1185846.

[13] Heloíse Manica, M. A. R. Dantas, and Murilo S. de Camargo. “An Architecture for Location-Dependent Semantic Cache Management”. In: *Proceedings of the Seventh International Conference on Enterprise Information Systems*. SciTePress: Science, 2005, pp. 320–325. DOI: 10.5220/0002524903200325. URL: http://dx.doi.org/10.5220/0002524903200325.

[14] Uber Technologies, Inc. *H3: A Hexagonal Hierarchical Spatial Index*. https://h3geo.org/docs/. 2025.

[15] Nicolas Drapier, Aladine Chetouani, and Aurélien Chateigner. “Enhancing Maritime Trajectory Forecasting via H3 Index and Causal Language Modelling (CLM)”. In: *Transactions on Machine Learning Research* (2024). DOI: 10.48550/ARXIV.2405.09596. URL: https://arxiv.org/abs/2405.09596.

[16] Open Geospatial Consortium. *OGC Filter Encoding 2.0 Encoding Standard*. Tech. rep. OGC 09-026r2. Open Geospatial Consortium, 2011.

[17] Shaoming Pan et al. “A Global User-Driven Model for Tile Prefetching in Web Geographical Information Systems”. In: *PLOS ONE* 12.1 (Jan. 2017). Ed. by Houbing Song, e0170195. ISSN: 1932-6203. DOI: 10.1371/journal.pone.0170195. URL: http://dx.doi.org/10.1371/journal.pone.0170195.

[18] Uber Technologies, Inc. *H3 API: Hierarchy Functions*. https://h3geo.org/docs/api/hierarchy/. H3 Project, 2025. URL: https://h3geo.org/docs/api/hierarchy/.

[19] Uber Technologies, Inc. *H3 Core Library: Overview*. https://h3geo.org/docs/core-library/overview/. H3 Project, 2025. URL: https://h3geo.org/docs/core-library/overview/.

[20] Uber Technologies, Inc. *H3 Core Library: Average Cell Area by Resolution*. https://h3geo.org/docs/core-library/restable/#average-area-in-km2. H3 Project, 2025. URL: https://h3geo.org/docs/core-library/restable/#average-area-in-km2.

[21] Uber Technologies, Inc. *H3 API: Region Functions*. https://h3geo.org/docs/api/regions/. H3 Project, 2025. URL: https://h3geo.org/docs/api/regions/.

[22] Uber Technologies, Inc. *H3 API: Grid Traversal Functions*. https://h3geo.org/docs/api/traversal/. H3 Project, 2025. URL: https://h3geo.org/docs/api/traversal/.

[23] Uber Technologies, Inc. *H3 Core Library: Resolution Table*. https://h3geo.org/docs/core-library/restable/. H3 Project, 2025. URL: https://h3geo.org/docs/core-library/restable/.

[24] Jeffrey Dean and Luiz André Barroso. “The tail at scale”. In: *Communications of the ACM* 56.2 (Feb. 2013), pp. 74–80. ISSN: 1557-7317. DOI: 10.1145/2408776.2408794. URL: http://dx.doi.org/10.1145/2408776.2408794.

[25] Nimrod Megiddo and Dharmendra S. Modha. "ARC: A Self-Tuning, Low Overhead Replacement Cache". In: *2nd USENIX Conference on File and Storage Technologies (FAST 03)*. San Francisco, CA: USENIX Association, Mar. 2003. URL: https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache.

[26] Hossein Pishro-Nik. *Basic Concepts of the Poisson Process*. 2014. URL: https://www.probabilitycourse.com/chapter11/11_1_2_basic_concepts_of_the_poisson_process.php.

[27] Debezium Community. *Debezium PostgreSQL Connector Documentation*. Debezium Project, 2025. URL: https://debezium.io/documentation/reference/stable/connectors/postgresql.html.

[28] Apache Software Foundation. *Apache Kafka Documentation*. https://kafka.apache.org/documentation/. Apache Kafka Project, 2025. URL: https://kafka.apache.org/documentation/.

[29] Vijay Nagarajan et al. *A Primer on Memory Consistency and Cache Coherence*. Springer International Publishing, 2020. ISBN: 9783031017643. DOI: 10.1007/978-3-031-01764-3. URL: http://dx.doi.org/10.1007/978-3-031-01764-3.

[30] Maurice P. Herlihy and Jeannette M. Wing. "Linearizability: a correctness condition for concurrent objects". In: *ACM Transactions on Programming Languages and Systems* 12.3 (July 1990), pp. 463–492. ISSN: 1558-4593. DOI: 10.1145/78969.78972. URL: http://dx.doi.org/10.1145/78969.78972.

[31] Werner Vogels. "Eventually consistent". In: *Communications of the ACM* 52.1 (Jan. 2009), pp. 40–44. ISSN: 1557-7317. DOI: 10.1145/1435417.1435432. URL: http://dx.doi.org/10.1145/1435417.1435432.

[32] *HTTP Caching*. June 2022. DOI: 10.17487/rfc9111. URL: http://dx.doi.org/10.17487/RFC9111.

[33] Google SRE Team. "Monitoring Distributed Systems". In: *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. URL: https://sre.google/sre-book/monitoring-distributed-systems/.

[34] John D. C. Little. "A Proof for the Queuing Formula: L = λW". In: *Operations Research* 9.3 (June 1961), pp. 383–387. ISSN: 1526-5463. DOI: 10.1287/opre.9.3.383. URL: http://dx.doi.org/10.1287/opre.9.3.383.

[35] Daniel S. Myers. *Lecture 12: The M/M/1 Queue*. https://pages.cs.wisc.edu/~dsmyers/cs547/lecture_12_mm1_queue.pdf. University of Wisconsin–Madison, CS 547 Lecture Notes [Online; accessed 22-October-2025]. 2017.

[36] Redis, Inc. *Key Eviction*. https://redis.io/docs/latest/develop/reference/eviction/. Redis, 2025. URL: https://redis.io/docs/latest/develop/reference/eviction/.

[37] Apache Software Foundation. *Introduction to Apache Kafka*. https://kafka.apache.org/24/getting-started/introduction/. Apache Kafka Project, 2024. URL: https://kafka.apache.org/24/getting-started/introduction/.

[38] Apache Software Foundation. *Apache Kafka 4.1: Design*. https://kafka.apache.org/41/design/design/. Apache Kafka Project, 2025. URL: https://kafka.apache.org/41/design/design/.

[39] Green Software Foundation. *Software Carbon Intensity (SCI) Specification and Guide*. https://sci-guide.greensoftware.foundation/. Green Software Foundation, 2025. URL: https://sci-guide.greensoftware.foundation/.